

JEUS

JDBC Guide

Version 3.0

We Speak in Technology

 ***Tmax Soft***

Document Number

Part Number : K:JSJD-1015-01-300

Last Revised : 3, December, 2001

Print :

Copyright.

Copyright©2001 Tmax Soft Co., Ltd. All Rights Reserved.

Tmax Soft Co., Ltd

18F Glass Tower, 946-1, Daechi-Dong, Kangnam-Gu

Seoul, Korea 135-708

This software and documents are made available only under the terms of the Tmax Soft License Agreement and may be used or copied only in accordance with the terms of this agreement. No

part of this document may be reproduced, transmitted, or translated in any form or by any means, electronic, mechanical, manual, or optical, without the prior written permission of

Tmax Soft Co., Ltd.

Trademark

Tmax, WebtoB, WebT, and JEUS are registered trademarks of

Tmax Soft Co., Ltd.

서 문

이 책은 JEUS JDBC의 개념과 프로그래밍하는 방법을 기술한다.

이 책의 구성

이 책은 JEUS JDBC에 대한 기본적인 개념을 습득하고 나서 간단한 예제를 통해서 프로그래밍 하는 방법을 터득한다.

■ I 장, JDBC 개념 및 드라이버 타입(Driver Type)

이 장에서는 JDBC에 대한 전반적인 개념 및 구조와 JDBC 드라이버 타입에 대해 설명한다.

■ II 장, JDBC 2.0확장 패키지의 특성

이 장에서는 JEUS에서의 제공하는 트랜잭션(Transaction) 및 연결 풀(Connection Pool)에 대해 설명한다.

■ III장, dbpooladmin을 이용한 connection pool 상태 관리

이 장에서는 JEUS에서 제공하는 dbpooladmin을 이용해 connection pool에 대한 상태에 대해 control할 수 있는 명령어에 대한 설명을 한다.

■ IV장, Sample Source를 통한 프로그램 구현

엔터프라이즈 어플리케이션을 이용하여 실제 프로그래밍에 대한 예제를 들어 설명한다.

누가 이 책을 읽어야 하는가

이 책은 JEUS JDBC를 사용하여, EJB와Servlet/JSP 프로그램에서 DB연동을 하려는 사람들에게 필요한 정보를 담고 있다.

글씨체에 대해

다른 대부분의 책과 같이 JAVA의 클래스 이름이나 소스 코드에서는 **Courier New** 글씨체를 사용하였다. 또한 주의할 사항은 **Note :** 와 같이 표시해 두었다.

관련된 책들

JEUS에 관한 매뉴얼은 다음과 같은 것들이 있다.

- **JEUS Installation Guide**

JEUS 시스템의 설치를 위한 준비 작업과 설치 요령, 설치 후의 처리 사항에 대해 설명하고 있는 매뉴얼이다.

- **JEUS Getting Started**

JEUS를 처음 설치한 후, EJB/Servlet Engine을 설정하고 Example bin들과, Servlet/JSP를 실행하는데 필요한 설명이다.

- **JEUS Administration Guide**

JEUS를 어떻게 관리 하는지에 관하여 설명하고 있다. JEUS의 전반적인 구조에 대해서 자세히 설명하며, 실제 JEUS 시스템을 구성하는 방법에 대해서 예를 들어가면서 설명한다. 이 책은 JEUS WebContainer Guide 및 JEUS Enterprise JavaBeans Guide 매뉴얼과 더불어 시스템 관리자가 꼭 읽어야 하는 책이다.

- **JEUS Administration Tool Guide**

JEUS 전체를 관리하고, 모니터링하며, JEUS 시스템에 관련된 configuration 파일의 생성을 도와주는 Administration Tool에 관한 내용을 다루고 있는 매뉴얼이다.

- **JEUS Deployment Tool Guide**

어플리케이션 프로그램 개발자가 작성된 EJB bin들로 EJB 모듈을 구성하거나 Servlet, JSP 프로그램과 함께 전체 시스템을 구성할 때 이들을 하나의 모듈로 packing하여 주고, packing 된 것을 시스템에 deploy하는 jdeploy Tool에 관한 내용을 다루고 있는 매뉴얼이다.

■ JEUS Client Container Guide

JEUS 서비스를 클라이언트가 사용할 수 있도록 기본 환경을 제공하는 Client Container의 구조와 사용 방법을 설명한다. Client Container는 기본적으로 JEUS의 네이밍 서비스, Scheduler, Security를 이용할 수 있는 환경을 제공한다.

■ JEUS Enterprise JavaBeans Guide

엔터프라이즈 자바 빈즈에 대한 설명과 JEUS의 EJB의 구조와 운영, 개발 등에 필요한 내용을 다루고 있는 매뉴얼이다.

■ JEUS JDBC Guide

JDBC에 대한 기본적인 개념과 DB Vendor별 Sample Programming을 기술하고 있다.

■ JEUS JNDI Guide

Java Naming & Directory Interface에 대한 설명과 JEUS JNDI를 사용하고 Naming Server를 설정하는 과정을 다루고 있는 매뉴얼이다.

■ JEUS JMS Guide

Java Messaging Service에 관한 설명과 JEUS JMS를 사용하고 설정하는 방법을 기술하고 있다.

■ JEUS JTS Guide

JEUS에서 제공되고 있는 Transaction 서비스의 이용 방법과 관련 설정 및 관리 방법을 설명하고 있는 매뉴얼이다.

■ JEUS Scheduler Guide

일정 간격으로 발생하는 이벤트를 처리 할 수 있는 JEUS Scheduler의 기본적인 개념과 간단한 예제를 제시하고 있다.

■ JEUS Security Guide

Security에 대한 일반적인 설명과 JEUS Security를 사용하고 설정하는 방법을 기술하고 있다.

■ JEUS Web Container Guide

JEUS Web Container의 관리와 이를 이용한 servlet, JSP 개발에 대한 내용을 담고 있다. Web Container 관리에 대해서는 모든 내용을 빠짐없이 기술하였고, 개발 측면에 대해서는 Servlet과 JSP의 기본적인 요소에 대한 설명과 각각의 구현 요소에 대해 간략히 설명하였다.

■ JEUS WebtoB Light Guide

JEUS에 내장된 WebServer인 WebtoB Light의 기능을 적절히 활용할 수 있도록

하기 위한 목적으로 만들어졌다. WebtoB Light의 전반적인 구조, 환경파일 작성 방법 및 WebServer의 관리 방법을 기술하고 있다.

■ **JEUS XML Guide**

XML의 개념과 JEUS에서 XML관련 Application을 사용하는 방법을 설명 하고 있다.

■ **JEUS CAS Guide**

CAS(Client Access Service)의 개념에 대한 기본적인 내용과 JEUS CAS의 세부적인 부분, 또한 JEUS CAS를 설정하고 사용하는 데에 대한 설명을 담고 있다.

글씨체에 대해

다른 대부분의 책과 같이 java의 클래스 이름이나 소스 코드에서는 **Courier New** 글씨체를 사용하였다. 또한 주의할 사항은 **Note :** 와 같이 표시해 두었다.

JEUS 제품별 기능

Edition	기 능
Base	WebtoB Light, Servlet, JSP, jadmin, JDBC Connection Pool, JEUS Manager, Security
Standard	JEUS Base, EJB, JTS(local transaction only), JMS, jdeploy, JNDI
Enterprise	JEUS standard, JTA(multi node, 이기종 DB)

차 례

서 문.....	2
이 책의 구성	2
누가 이 책을 읽어야 하는가.....	3
글씨체에 대해	3
관련된 책들	3
글씨체에 대해	5
JEUS 제품별 기능	5
I . JDBC 개념 및 드라이버 타입 (DRIVER TYPE)	8
1. JDBC 개념	8
2. JDBC 구조	8
3. JDBC 드라이버 타입(DRIVER TYPE).....	9
II . JDBC 2.0확장 패키지의 특성.....	13
1. JNDI FOR NAMING DATABASES	13
2. 연결 풀링(CONNECTION POOLING)	13
3. 분산 트랜잭션 지원	13
4. JNDI와JDBC API	14
5. 데이터 소스(DATA SOURCE)	14
5.1 Data Source Properties.....	15
5.2 데이터소스 이름(Data source name).....	16
5.3 데이터소스 구현	16
6. 연결 풀링(CONNECTION POOLING)이란?.....	18
6.1 연결 풀링 장점	19
6.2 연결 풀을 관리하는 방법	20
7. 트랜잭션(TRANSACTION)	20
7.1 분산 트랜잭션 데이터소스	21
8. JNDI와 제우스 연결 풀	22
8.1 연결 풀 구성	23
9. VENDOR SPECIFIC DATASOURCE PROPERTY	26
9.1 Oracle.....	26
9.2 Sybase	27
9.3 db2	28
9.4 MS-SQL.....	31

10. 트랜잭션을 이용한 무상태 세션 빈 예제	31
10.1 Oracle.....	32
10.2 Sybase	32
10.3 db2	33
10.4 MS-SQL	33
11. 톨을 통한 연결 풀 구성.....	34
11.1 데이터 소스 명령.....	35
III . DBPOOLADMIN을 이용한 CONNECTION POOL 상태 관리.....	36
1. DBPOOLADMIN 에서 CONNECTIONPOOL 정보를 알려주는 명령어	38
1.1 console인 경우.....	38
1.2 administration Tool인 경우	39
2. DBPOOLADMIN 에서 CONNECTIONPOOL 상태를 CONTROL 하는 명령어.....	42
2.1 console	42
2.2 Administration Tool	43
IV .SAMPLE SOURCE를 통한 프로그램 구현.....	44
1. CONNECTIONPOOLDATASOURCE에 의한 STATEFUL SESSION BEAN	45
1.1 Oracle.....	45
2. LOCALXDATASOURCE에 의한 STATEFUL SESSION BEAN	48
2.1 db2	48
3. XDATASOURCE에 의한 STATEFUL SESSION BEAN	52
3.1 Sybase&Oracle.....	53

I. JDBC 개념 및 드라이버 타입 (Driver Type)

이 장에서는 JEUS 에서 데이터베이스에 어떻게 연결하고, 그 연결을 관리하는지에 대해서 알아본다. 또한 JDBC(Java Database Connectivity)에 대해서 알아본다.

1. JDBC 개념

JDBC(Java Database Connectivity)는Java 응용프로그램(Application)이 관계형 데이터베이스(relational database)에 접속하기 위한 Java Standard API이다. Sun Microsystems Inc에서 만든 이 스펙(specification)은 JDBC Vendor로 하여금 표준을 구현 및 확장하도록 해준다.

JDBC는X/Open SQL Call Level Interface를 근간으로 하고 있으며, 현재 스펙3.0을 발표한 상태이다. 제우스(JEUS)는JDBC 스펙2.0의 확장 팩에서 규정되어 있는 여러 Feature를 제공한다.

2. JDBC 구조

JDBC는 자바 인터페이스를 이용하여 데이터베이스 연동을 수행하고, 이들 인터페이스들은 각각 서로다른 개별적인 제공업체들에 의해 구현(implement)되었다. 특정한 데이터 베이스 엔진을 위해 JDBC 인터페이스를 구현하는 클래스들을 JDBC 드라이버라고 한다.

JDBC에는 각각의 데이터베이스에 특징적인 부분이 모두 숨겨져 있으며 사용자는 단지 자신의 애플리케이션에만 신경쓰면 된다.

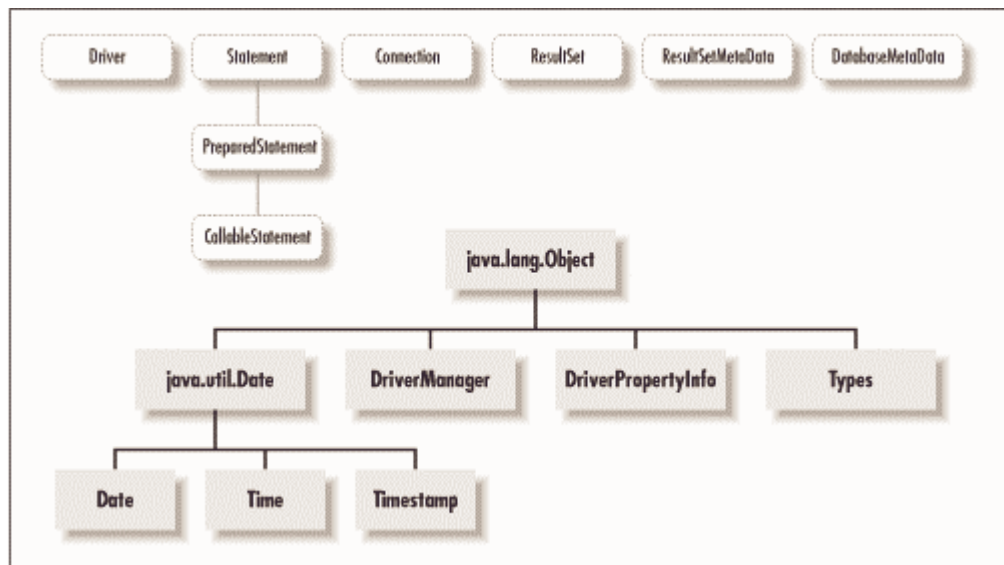


그림1-1 JDBC API 패키지 java.sql에 있는 클래스와 인터페이스들

3. JDBC 드라이버 타입(Driver Type)

JDBC 드라이버는 4가지 범주로 나눌 수 있다. 이들 범주에 속하는 드라이버는 각각 자신만의 장점을 가지고 있으며, 수행 속도라는 측면에서 고려해 볼만한 가치가 있다. 아래의 각 드라이버 타입들을 살펴보고, 응용프로그램에서 사용할 JDBC 드라이버를 선택할 때 이들 차이점을 신중히 고려해야 한다.

■ 타입1 JDBC-ODBC Bridge

이 드라이버는 표준 드라이버 호출들을 대응되는 ODBC 호출로 변환하고, 운영 시스템의 ODBC 라이브러리를 통해 ODBC 데이터소스로 전송한다. 즉 JDBC 호출을 ODBC 호출로 변환하고, 이를 ODBC Driver로 보낸다.

타입1 JDBC driver를 사용하려면 client 컴퓨터에 ODBC driver와 JDBC driver가 설치되어 있어야 한다. JDBC를 사용해서 ODBC 데이터 소스에 접근하는 것은 매우 비효율적이고, 제한이 많다. 시스템이 여러 단계를 거쳐 데이터베이스 호출을 전달할 뿐만 아니라 JDBC의 기능은 어떤 ODBC 드라이버라도 다룰 수 있는 것으로만 제한된다. 이로 인해 동적으로 드라이버를 다운로드하여 사용하는 환경에서는 사용하기 어려우며 고정된 드라이버가 설치되어 운영되는 환경에 적합하다.

sun에서 JVM에 기본적으로 제공하는 JDBC-ODBC driver는 sun.jdbc.odbc.Jdbc OdbcDriver이다.

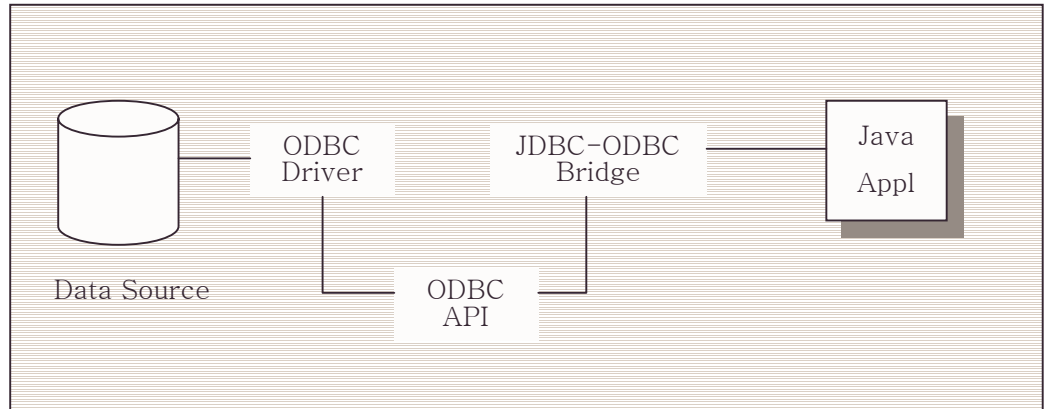


그림1-2 type1 JDBC-ODBC Bridge

■ 타입2 native-API partly-Java driver

타입 2는 일부가 Java로 되어 있는 JDBC Driver를 의미한다. 즉 자바 드라이버를 사용해서 특정 업체의 API와 상호 통신한다. 이것은 타입1드라이버와 개념이 비슷하고, ODBC API로의 변환단계가 생략된 것을 제외하고 다른 점이 없다. JDBC를 사용해서 데이터베이스 호출이 만들어지면 드라이버는 해당 요청을 벤더의 API가 이해할 수 있는 형태로 바꾸고, 데이터베이스가 요청을 처리한 다음, 결과를 API를 통해 돌려준다. 이때 JDBC드라이버는 JDBC 표준을 따르는 결과를 만들어 응용 프로그램에 리턴한다. 따라서 타입3와 마찬가지로 클라이언트 컴퓨터에 설치되어 있는 로컬 드라이버가 필요하다.

다만 이 로컬 드라이버는 ODBC 형식이 아니고 데이터베이스를 만든 벤더가 제공하는 다른 형태의 드라이버이다. 타입2 역시 벤더의로컬 드라이버와 JDBC 드라이버가 클라이언트 컴퓨터에 설치되어있어야 사용가능하다.

이러한 형식은 Oracle, Informix, DB2 등 DBMS 벤더들이 제공하는 API를 다시 호출하게 되는데 Type 1 과 마찬가지로 Client 컴퓨터에 정적으로 driver를 설치하여 데이터베이스와 연동하는 환경에적합하다.



그림1-3 type2 JDBC 드라이버

■ 타입3 net-protocol all java driver

타입3는 완전한 Java 기술 및 코드로 이루어진 JDBC Driver를 의미한다.

이 타입은 JDBC API 표준에 의하여 만들어 졌기 때문에 DBMS의 종류에 상관없이 사용할 수 있다.

DBMS 독립적인 net-protocol을 사용하여 미들웨어 계층의 서버와 통신하며, 이 미들웨어 서버는 JDBC 호출을 각 특정 벤더 DBMS 호출로 변환한다. 이때 미들웨어 서버는 게이트웨이(gateway) 역할을 한다. 4가지 타입 중에서 가장 융통성이 뛰어난 드라이버 타입이기 때문에 동적으로 다운로드되는 애플릿(Applet) 같은 환경에 더욱 적합하며, 클라이언트가 정적으로 이루어졌다고 하더라도 타입3를 사용하면 시스템 유지보수가 더욱 쉬워진다.

따라서 특별한 요구사항이 필요한 경우 즉, 특정 DBMS 만에 제공하는 기능을 사용할 필요가 있는 경우 또는 DBMS 자체가 타입3를 지원하지 않는 경우를 제외하고는 타입3를 사용하는 것이 가장 적절하다. 이와 더불어 문제점이라고 한다면, 데이터베이스와 대화하기 위해 미들웨어 서버가 타입1이나 타입2를 사용할 지도 모른다는 것이다.



그림1-4 type3 JDBC 드라이버

■ Type4 native-protocol all java driver

타입 4 또한 완전한 Java 기술 및 코드로 이루어진 JDBC Driver 를 의미한다.

하지만 타입3와는 다르다. JDBC API 표준을 기준으로 만들었다 기 보다는 DBMS 벤더가 표준을 기준으로 기능을 추가 또는 삭제 하여 독자적인 형태로 만든 타입이다.

따라서 특정 DBMS에 의존적인 반면에 해당 DBMS만이 제공하는 기능들을 사용할 수 있다는 장점이 있다.

즉4가지 타입의 드라이버 중 데이터베이스에 가장 효율적으로 접근 방법을 제공한다. 또한 추가로 설치해야 할 라이브러리(library)나 미들웨어가 필요 없기 때문에 배포하기에도 가장 손쉬운 방법을 제공한다. 따라서 타입4는 클라이언트가 동적인 환경에서 특정 DBMS만이 제공하는 기능들을 사용할 경우 적합하다.

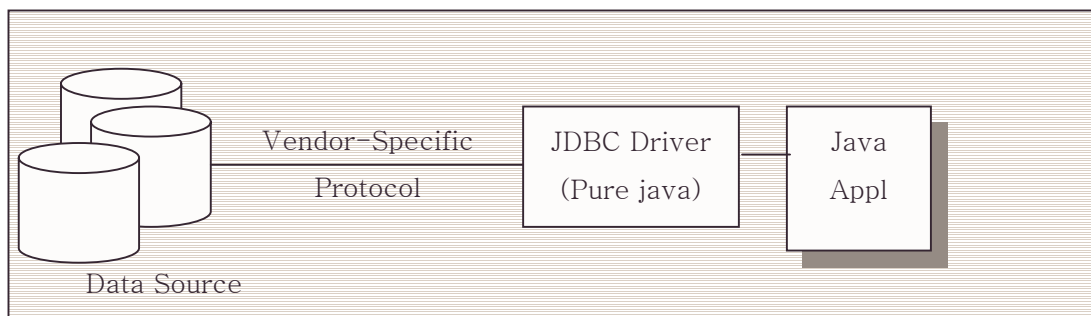


그림1-5 type4 JDBC 드라이버

II. JDBC 2.0확장 패키지의 특성

JDBC 스펙(specification) 2.0으로 오면서 스펙에는 많은 변화가 있었다. 그 중 대표적인 것은 아래와 같다.

1. JNDI for naming databases

데이터소스와 데이터베이스에 대한 연결을 관리 하는데 JDBC 드라이버 관리자(DriverManager)와 더불어 JNDI(Java Naming and Directory Interface)를 사용할 수 있다.

응용 프로그램이 JNDI를 사용한다면, JNDI로부터 얻은 이름은 데이터베이스에 접근하기 위한 JDBC 드라이버와 특정 데이터베이스 인스턴스를 명시하는 논리적 이름, 즉 별명(alias)이다. 개발자는 데이터베이스에 접근하는데 JNDI를 사용함으로써 특정 JDBC 드라이버나 JDBC URL과 무관하게 코딩할 수 있다.

2. 연결 풀링(Connection Pooling)

JDBC 2.0확장 팩API는JDBC 드라이버 상에서 연결 풀(Connection Pool)을 구현할 수 있도록 ‘hooks’를 명시하고 있다. 데이터베이스 연결을 생성하고 끊는 것은 많은 시간과 노력을 필요로 하기 때문에, 좋은 성능을 얻기 위해서 특히 서버 응용프로그램에 대해서는 매우 중요하다.

3. 분산 트랜잭션 지원

JDBC 드라이버는 Java Transaction API(JTA)에 의해서 사용되는 표준2 phase-commit protocol을 지원함으로써 분산 트랜잭션을 지원한다. 개발자들은 JDBC 드라이버가 제공하는 분산 트랜잭션을 이용함으로써 다중 데이터베이스 서버에 걸쳐 트랜잭션을 요구하는 Enterprise JavaBeans를 개발

할 수 있다.

4. JNDI와JDBC API

JNDI(Java Naming and Directory Service)는 어플리케이션으로 하여금 네트워크를 통하여 원격 자원에 대한 서비스를 찾아 사용할 수 있는 방법을 제공한다. JDBC 응용 프로그램은 특별히 데이터베이스 서비스를 이용한다. 대부분의 JNDI를 통해서 이용되는 원격 서비스는 엔터프라이즈 서비스인 경우가 많다. 데이터베이스 서비스 또한 예외는 아니다. 데이터베이스 서비스를 이용하는 데 있어 JNDI를 사용하는 것은 자원 관리의 용이성 뿐만 아니라 어플리케이션 코드 관리에도 많은 도움을 준다.

5. 데이터 소스(Data Source)

일반적으로 데이터 소스는 데이터를 저장하는 장소(repository)를 의미한다. 데이터 소스는 로컬 파일에서부터 원격 서버에 있는 대용량 데이터베이스일 수도 있다. 여기서는 데이터 소스의 의미를 데이터베이스에 국한시켜 설명한다.

JDBC 드라이버에서는 데이터 소스에 매핑되는 데이터소스(DataSource)객체(Object)가 존재한다. 응용 프로그램은 연결(Connection)을 이용하여 데이터 소스에 접근하고, 데이터소스 객체는 특정 데이터소스 DataSource인스턴스가 나타내는 연결에 대한 팩토리(factory)라고 볼 수 있다.

DataSource 객체를 사용하는 것은 데이터 소스에 대한 연결을 구성하는데 있어서 DriverManager보다 더 선호하는 방식이다.

DataSource 인터페이스와 DriverManager 클래스 사이에는 유사한 점이 많다.

그러나 가장 큰 차이점을 들자면 아래와 같다.

- DataSource 객체는 데이터 소스를 명시하고 설명하는 특성(property)를 갖고 있다
- DataSource 객체는 이를 사용하는 응용프로그램과는 무관하게 JNDI Naming Service를 사용하여 생성되고, 배치되고, 관리된다.

JNDI Naming Service를 사용함으로써 DataSource 객체가 DriverManager

에 대해 갖는 장점은 크게 두 가지이다.

첫째는 응용프로그램이 드라이버에 대한 정보 DriverManager는 필요로 하지만 알 필요가 없다. 즉hard coding을 필요로 하지 않는다. 개발자나 시스템 관리자는 데이터 소스에 대한 논리적 이름을 선택해서 JNDI Naming Service에 등록한다. 응용 프로그램은 단지 등록된 논리적 이름을 사용하고, Naming Service는 논리적 이름에 대응하는 DataSource 객체를 제공한다. 그러면, 데이터 소스에 대한 연결을 생성하는데 DataSource 객체를 사용할 수 있다. 두 번째는 시스템 개발자로 하여금 DataSource 클래스가 연결 풀(connection pool)이나 분산 트랜잭션(distributed transaction)같은 특징의 잇점을 취할 수 있도록 구현할 수 있게 한다.

데이터소스를 배치(deploy)하는 과정은 크게 세가지로 나눌 수 있다.

- DataSource 클래스의 인스턴스를 생성한다.
- 인스턴스의 특성을 설정한다.
- JNDI API를 사용하는 Naming Service에 인스턴스를 등록한다.

5.1 Data Source Properties

데이터소스(DataSource)객체는 실제 세계와 매핑되는 데이터 소스(data source)를 명시하고 설명하는 프라퍼티(property)들을 가지고 있다. 이들 프라퍼티는 데이터베이스 서버의 위치, 데이터베이스 이름 또는 그 서버와 통신하기 위한 프로토콜 같은 정보를 포함한다. JDBC 스펙2.0에 프라퍼티를 명시하고 있긴 하지만, 하나의 프라퍼티 – description –을 제외하고 강제되는 것은 없다. 달리 말하면 데이터소스 객체가 아래 정의되어 있는 표준 프라퍼티를 모두 구현 지원하지 않아도 무방하다. 따라서, 각 드라이버 벤더마다 설정해야 되는 프라퍼티는 달라질 수 있다.

아래는 스펙에 정의되어 있는 표준 데이터소스 프라퍼티 이다.

프라퍼티 이름	타입	설명
databaseName	String	서버상의 특정 데이터베이스 이름
dataSourceName	String	XADataSource나ConnectionPoolDataSource에 대한 논리적 이름. 연결 풀이나 분산 트랜잭션이 구현되었을 경우에 한해서만 사용된

		다.
description	String	데이터 소스에 대한 설명
networkProtocol	String	서버와 통신하기 위해 사용되는 프로토콜
password	String	데이터베이스의 사용자 비밀번호
portNumber	int	데이터베이스 서버의 리스닝 포트 번호
roleName	String	초기 SQL rolename
ServerName	String	데이터베이스 서버 이름.(hostname)
user	String	사용자 Account Name

표2-1 데이터소스 프라퍼티

5.2 데이터소스 이름(Data source name)

JDBC API는 데이터 소스에 JNDI 이름을 연결할 때 간단한 규칙을 사용한다.

모든 데이터 소스는 반드시 JNDI 이름공간(namespace)의 jdbc naming subcontext 또는 그 아래 subcontext에 등록해야 한다. jdbc subcontext의 위는 루트 이름 공간이다.

5.3 데이터소스 구현

DataSource 인터페이스는 3가지 다른 종류의 연결을 제공하도록 구현될 수 있다.

DataSource 객체가 이름 서비스를 사용함에 따라 데이터소스 객체가 만든 모든 연결은 이식성과 쉬운 관리의 잇점을 제공한다.

ConnectionPoolDataSource 및 XADataSource 인터페이스 구현한 클래스와 함께 작동하는 DataSource를 구현한 클래스는 연결 풀이나 분산 트랜잭션을 제공한다. 다음은 DataSource 인터페이스를 구현하는 3가지 일반적인 클래스 부류를 요약한 것이다.

- 기본 데이터소스 클래스
- 드라이버 벤더에서 제공하며, 풀링이나 분산 트랜잭션을 제공하지 않

는 표준 연결(Connection)객체를 생산한다. 이식성과 손쉬운 유지관리가 장점이다.

- 풀링을 지원하는 데이터소스 클래스
- 드라이버 벤더나 응용프로그램 서버에서 제공하며, 연결 풀링(Connection Pooling)에 참여하는 연결 객체(Connection Object) 즉 recycled될 수 있는지를 생산한다. 기본 데이터소스 클래스에서 제공하는 장점이외에 성능 향상을 제공한다. ConnectionPoolDataSource 클래스와 함께 작동한다.
- 분산 트랜잭션을 지원하는 데이터소스 클래스
- 응용프로그램 서버(예: EJB Server Vendor)에서 제공하며, 분산 트랜잭션에 사용할 수 있는 연결 객체 두 개 이상의 DBMS 서버에 접근하는 트랜잭션을 생산한다. 물론 연결 풀링을 기본적으로 제공한다. XADataSource 클래스와 함께 작동한다.

데이터소스(DataSource) 인터페이스를 구현한 클래스의 인스턴스는 특정 데이터소스를 의미한다. 이 인스턴스에 의해 만들어진 모든 연결은 같은 데이터 소스를 참조한다.

기본 데이터소스 구현에 있어서, DataSource.getConnection() 함수에 의해 반환되는 Connection 객체는 DriverManager에 의해 만들어지는 Connection 객체와 동일하다. 즉, 둘 다 데이터 소스에 대한 물리적 연결을 나타낸다.

연결 풀을 구현한 데이터소스 객체는 위와 비슷하게 그 데이터소스 객체가 나타내는 특정 데이터 소스에 대한 연결을 제공한다. 그러나, 반환되는 Connection 객체는 약간 의미가 다르다. getConnection() 함수에 의해 반환되는 Connection 객체는 실제 물리적 연결이 아니라 PooledConnection 객체의 핸들(handle)이다. 이 객체를 사용하는 응용 프로그램은 평상시 그대로 이 Connection 객체를 사용한다. 즉 응용 프로그램 개발자는 전혀 Connection 객체에 대해 인식할 수 없다. 다만 코딩에서 달라지는 부분이 있다면, 명시적으로 프로그램 코드내에서 이 연결을 닫아야 한다는 것이다.

데이터소스를 또한 분산 트랜잭션 환경에서 작동할 수 있도록 구현할 수 있다. 예를 들면, EJB 서버는 분산 트랜잭션을 지원하고, 이와 상호 작동할 수 있도록 구현된 데이터소스를 요구한다. 이 경우, getConnection()에 의해 반환된 Connection 객체는 분산 트랜잭션에 사용된다. 연결 풀과 마찬가지로

가지로, 트랜잭션 관리는 내부적으로 이루어진다. 따라서 분산 트랜잭션을 사용하는 것은 쉽다. 단지 분산 트랜잭션을 사용할 때, 프로그램 코드내에 commit 이나 rollback함수를 사용할 수 없다. 또Connection 객체에 auto-commit mode를 설정할 수 없다. 이런 제한은 이를 관리하는 별도의 트랜잭션 관리자를 EJB 서버에 제공하기 때문이다.

JEUS에서는 위의 모든 데이터소스 타입을 제공한다.

6. 연결 풀링(Connection Pooling)이란?

JDBC 2.0확장 API에서 연결 풀링은 데이터베이스 연결 캐싱에 대한 Framework이다. 연결 풀링은 물리적인 연결을 재사용할 수 있도록 해주며, 어플리케이션에 대한 부하를 감소시킨다.

아래는 이를 이루고 있는 핵심 개념이다.

- Connection pool data sources
- 개념이나 기능적인 면에서 data sources와 비슷하나, data source가 일반적인 connection instance를 반환하는 반면 이것은 pooled connection instance를 반환한다.
- Pooled connection
- 데이터베이스와의 하나의 물리적인 연결을 나타내는 pooled connection instance이다. 이 연결은 논리적인 연결 인스턴스(logical connection instance)에 사용되는 동안 DB와 연결을 계속 유지한다.

논리적 연결 인스턴스는 pooled connection instance에 반환되는 단순한 연결 인스턴스이다.(예를 들면, 단순한 Connection Instance). 각 논리적 연결 인스턴스는 pooled connection 인스턴스로 나타나는 물리적 connection에 대해 일시적인(temporary) 핸들로서 작동한다.

JDBC 1.0은 관계형 데이터베이스에 액세스를 위해 Java API를 정의한다. JDBC 2.0의 도입으로, API는 다음 두 부분으로 나누어진다.

JDBC 2.0코어 API	여기에는 발전된 개선 내용이 포함되어 있으나 용이한 사용을 위해 JDBC 1.0 API처럼 작게 유지한다. 1.0 API에 대해 작성된 코드는 2.0 API로 작업을 계속한다. 2.0 API 클래스는 java.sql 패키지에 남아 있다.
JDBC 2.0표준 확장 API	벤더가 기능을 제공할 준비가 되어 있고 프로그래머가 기능을 사용할 준비가 되어 있는 경우, 특정 종류의 추가 기능을 정의한다.

표2-2 JDBC 2.0 API

연결 풀링은 JDBC 2.0 표준 확장 API의 일부로 정의된다. 표준 확장 API의 또 다른 부분에서는 관계형 데이터 서버에 액세스하는 DriverManager 오브젝트를 사용하는 대신 JNDI(Java Naming and Directory Interface) 및 데이터 소스 오브젝트의 사용을 제공한다.

6.1 연결 풀링 장점

자원(서블릿, EJB 또는 어플리케이션)이 데이터베이스에 접근하려고 시도할 때마다, 해당 데이터베이스에 연결해야 한다. 데이터베이스 연결은 오버헤드를 야기한다. 연결 풀링은 자원에 대한 연결을 생성하고 유지한 후 연결이 더 이상 필요하지 않을 때 해제하도록 요구한다.

웹 사용자는 자주 연결 및 해제하므로 웹 기반 응용프로그램의 경우 특히 데이터베이스 연결과 관련된 오버헤드가 많다. 그리고, 사용자와의 상호작용은 인터넷의 서핑 특성상 일반적으로 더 적다.

어떤 경우에는 사용자와의 상호작용 중에 소비하는 것보다 더 많은 노력을 연결하고 연결 해제하는 데 소비한다. 더우기, 인터넷 요청은 가상적으로 어디에서나 도착할 수 있으므로, 사용 볼륨은 크고 예견하기 어려울 수 있다.

이 같은 문제점을 해결하기 위해, 제우스에서 응용프로그램에 의해 공유되는 데이터베이스 연결 풀을 제공한다.

관리자는 연결 풀링을 통해 웹 기반 응용프로그램에서 사용되는 자원을 제어하고 줄일 수 있다. 연결 풀은 자원을 보존하며, 여러 사용자 요청 간의 연결 오버헤드를 전파한다. 또한 연결 풀링은 웹 기반 응용프로그램의 응답 시간을 향상시킨다.

사용자가 웹에서 웹 어플리케이션(서블릿, EJB)을 요청을 할 때, 어플리케이션은 데이터소스에 접근한다. 데이터 소스가 풀에서 기존 연결을 찾아 사용하므로, 사용자 요청은 새 연결을 작성하는 오버헤드를 주지 않는다.

각 연결은 특정 사용자 요청과 관련되어 있다. 요청이 만족되고 응답이 사용자에게 리턴되면, 데이터 소스는 연결을 재사용할 연결 풀로 리턴한다. 연결해제 오버헤드를 피할 수 있다

각 사용자 요청은 얼마되지 않는 연결 또는 연결해제 비용을 지불한다. 초기 자원이 풀에서 연결을 생성하기 위해 사용된 후에, 기존 연결이 재사용되므로 추가 오버헤드는 얼마되지 않는다.

6.2 연결 풀을 관리하는 방법

제우스에서는 아래와 같은 방법으로 연결 풀을 설정하고 유지한다.

연결이 설정되면, 사용자 요청에 응답하여 연결을 분배함으로써 관리한 후 하우스키핑 작업을 수행하여 사용 가능한 연결과 연결 수요 간의 균형을 유지한다. 이것은 서블릿이나 어플리케이션이 연결을 필요로 할 때 기존의 연결이 사용 가능함을 보장한다.

예를 들어, 연결 풀은 주기적으로 유헤 또는 버려진 연결을 구분한다. 유헤 연결을 종료하고 연결 풀로 버려진 연결을 반환한다. 이것은 연결 수요가 낮을 때 더 적은 연결이 사용 가능함(그리고 더 적은 자원이 사용됨)을 의미한다.

유헤 연결은 데이터소스의 유헤 제한시간 특성에 지정된 시간에 사용되지 않은 연결이다. 연결은 소유하는 서블릿이나 어플리케이션이 종료하거나 응답하지 않을 때 버려진다.

7. 트랜잭션(Transaction)

데이터베이스 작업의 단위를 말한다. 이는 작업의 일관성과 관계가 있다.

다음은 말하는 트랜잭션에 대한 정의이다.

트랜잭션은 하나의 논리적 작업 단위로 수행되는 일련의 작업입니다.

작업의 논리적 단위는 ACID(원자성, 일관성, 격리성 및 영속성) 속성이라고 하는 네 가지 속성을 통해 트랜잭션으로서의 자격을 부여한다.

■ 원자성

트랜잭션은 더 이상 분류할 수 없는 작업 단위여야 하며 모든 데이터 수정 작업이 수행되거나 하나도 수행되지 말아야 한다.

■ 일관성

완료된 트랜잭션의 모든 데이터는 일관적이어야 한다.

관계형 데이터베이스에서는 트랜잭션 수정에 모든 규칙을 적용하여 모든 데이터 무결성을 유지해야 한다.

■ 격리성

동시 트랜잭션에 의한 수정은 다른 동시 트랜잭션에 의한 수정과 격리되어야 한다.

트랜잭션에서 다른 동시 트랜잭션이 수정하기 전 상태의 데이터를 보거나, 두 번째 트랜잭션이 완료된 후의 데이터를 볼 수는 있지만 중간 상태는 볼 수 없다.

결과적으로 시작 데이터를 다시 로드하고 일련의 트랜잭션을 재생하여 원래 트랜잭션이 수행된 후의 상태로 데이터를 되돌릴 수 있는데 이를 순차성이라고 한다.

■ 영속성

트랜잭션이 완료되고 나면 그 영향이 영구적으로 시스템에 적용된다. 수정은 시스템에 오류가 발생한 경우에도 지속된다.

자세한 내용은 *JTS*를 참고 하기 바란다.

7.1 분산 트랜잭션 데이터소스

XADataSource는 분산 트랜잭션에 사용될 수 있는 PooledConnection으로부터 유도된 XAConnection 객체를 만든다. 분산 트랜잭션을 사용하는 응용 프로그램은 XADataSource와XAConnection을 구현한 JDBC 드라이버를 통하여 수행되어야 한다.

로컬 트랜잭션은 하나의 데이터베이스로 보내지는 일련의 명령어들이다. 이 명령들은 단위 작업(Unit of Work)으로 모든 작업이 commit 되거나 rollback이 되어야 한다. 트랜잭션이 commit된다면, 데이터베이스에 대한 모든 변화가 commit 된다. 즉 데이터베이스 입장에서 생각하면 Log에 기록된 데이터에 대한 모든 변화가 데이터에 적용되고, 기록이 지워짐을 의미한다. 반대로 rollback은 단지 log 기록이 데이터에 적용되지 않고 로그에서 지워진다. 분산 트랜잭션은 일련의 명령어들이 분산된다. 즉, 이들은 하나 이상의 데이터베이스 또는 자원 관리자(resource manager) – 트랜잭션 가지(transaction branch)를 이루는 –로 보내진다. 분산 트랜잭션이 commit 되기 위해서는 이에 참여하는 모든 자원 관리자가 commit 하는데 찬성해야 한다. 자원 관리자중 하나라도 반대한다면, 이 분산 트랜잭션에 참여한 모든 자원 관리자는 그 시점까지의 모든 변화를 rollback 한다.

XADataSource는DataSource와ConnectoinPoolDataSource 인터페이스와 매우 유사하다. 이들의 차이점은 그들이 만들어내는 connection의 종류가 다르다는 점이다. DataSource 객체는 데이터 소스에 대한 물리적인 연결을 나타내는 Connection을 객체를 만든다. ConnectionPoolDataSource는 데이터 소스에 대한 재사용 가능한 물리적인 연결을 만드는 PooledConnection 객체를 생산한다. XADataSource 객체는 분산 트랜잭션에 사용될 수 있는 재사용 가능한 물리적 연결을 만드는 XAConnetion객체를 생산한다. PooledConnection 객체와 XAConnection객체가 생산하는 Connection 객체는 그들이 나타내는 하단의 물리적 연결에 대한 핸들이다. 분산 트랜잭션을 수행하기 위해서는 응용 프로그램은 XAConnection에서 만든 Connection 객체를 얻어야 한다. 응용 프로그램은 Connection 객체를 얻기 위해서 단지 DataSource.getConnection 함수만 호출하면 된다.

트랜잭션 Recovery 관련하여 Connection Pool 설정시 주의할 것은 user/password 설정이다.

Transaction Manager 가 죽었을 경우는 Transaction Manager 가 자동으로 트랜잭션 recovery 를 수행하고, 데이터베이스가 죽었을 경우에는 관리자가 Dbpooladmin을 이용하여 recovery 과정을 시작시킨다.

두 경우 모두 Connection Pool 에 설정된 User/Password 를 이용하여 connection 을 얻어 그것으로부터 트랜잭션 관련 정보를 얻어온다. 이 때 데이터베이스에 따라 트랜잭션 관련 정보를 사용할 수 있는 권한 설정이 다른데 Connection Pool 에 사용된 User가 이 정보를 사용할 수 있는 권한이 있어야 한다. 가장 간단하게는 이 User 에게 DBADMIN 의 권한을 주는 것인데 이 경우는 User/Password 가 설정파일에 보여 보안상 위험할 수 있다. 각 데이터베이스의 트랜잭션 recovery 정보 참조 관련 설정을 참조하여 설정한다.

8. JNDI와 제우스 연결 풀

기본적으로 어플리케이션이 JDNI를 이용하여 연결 풀을 찾을 때 마다 해당 클라이언트의 JVM에 연결 풀이 새로 생성된다. 다시 말해 만약3개의 클라이언트가 서로 다른3개의 JVM에서 각각 연결 풀을 JNDI로부터 찾아오면 각각의 JVM에 같은 구성을 갖는 경우, 독립적인 연결 풀이 별도로 생성된

다. 만약 같은 JVM내에서 클라이언트가 두 번 이상 같은 연결 풀을 JNDI로부터 찾으면 처음에는 해당 연결 풀이 생성되고, 두 번째부터는 새로 생성되는 것이 아니라 단지 이미 생성된 연결 풀의 참조자를 반환한다. 만약 EJB 엔진상의 EJB 빈에서 연결을 얻는다면, 빈이 배치될 때 연결을 얻는 것이 아니라 EJB 빈에서 최초 연결을 시도할 시에 연결 풀을 생성한다.

8.1 연결 풀 구성

연결 풀(Connection Pool) 구성에 관한 설정은 구성파일 중JeusMain.xml에서 이루어 진다. 각 데이터베이스가 제공하는 JDBC driver가 요구하는 데이터소스의 프라퍼티가 모두 다르므로 값을 설정할 때 세심한 주의가 필요하다.

아래는 연결 풀에 관련된 JeusMain.xml의 요소들이다.

- Vendor

데이터베이스 벤더이름

- ExportName

JNDI에 등록시 사용될 이름.

- DatabaseName

데이터베이스 이름. 각 벤더마다 사용하는 의미가 다를 수 있다.

Oracle의SID값, DB2, Sybase일 경우 실제 DB의 이름. 이 값을 설정할 때는 DB 관리자에 물어서 설정한다.

- DataSource Class Name:

실제 jdbc driver package에 들어있는 Driver Vendor의class 이고, 이것은 반드시 설정 되어야 한다.

- 생성하고자 하는 DataSource 형태에 따라 달라진다.

- 기본 데이터소스

For Oracle : oracle.jdbc.pool.OracleDataSource

For Sybase : com.sybase.jdbc2.jdbc.SybDataSource

For db2: COM.ibm.db2.jdbc.DB2DataSource

- 풀링을 제공하는 데이터소스

For Oracle :

oracle.jdbc.pool.OracleConnectionPoolDataSource

For Sybase:

com.sybase.jdbc2.jdbc.SybConnectionPoolDataSource

For db2:

COM.ibm.db2.jdbc.DB2ConnectionPoolDataSource

- 분산 트랜잭션을 제공하는 데이터소스

For Oracle : oracle.jdbc.xa.client.OracleXADataSource

For Sysbae : com.sybase.jdbc2.jdbc.SybXADataSource

- For db2: COM.ibm.db2.jdbc.DB2XADataSource

- DataSourceName

Driver에 따라 설정이 필요한 것이 있고 아닌 것이 있다.

이것은 driver의 설정 요구 사항에 specific한 것으로 대개는 DataSource Class Name과 같은 값이 설정된다.

- Service Name

inet의SID 값을 지정하는것으로 이 값을 설정할 때는 DB관리자에 물어서 설정한다.

- Description

데이터 소스에 대한 설명을 적는다. Oracle,db2의 경우 생략이 가능하나, Sybase의 경우에는 반드시 적어야 한다.

- NetworkProtocol

Connection에서 사용하게 될 프로토콜, 일반적으로 기본값을 사용하므로 설정하지 않아도 무관한다. 설정하려면 각 벤더사가 제공하는 JDBC 드라이버 참조서를 참고한다. **Sysbase일 경우, 반드시 Tds이**

어야 한다.

- Password : 사용자의 패스워드
- PortNumber : 연결하고자 하는 db instance의listener 포트 번호
- ServerName : db가 설치되어 있는 노드 이름
- User : 사용자의 이름
- DriverType : 사용할 jdbc 드라이버 타입, oracle에서만 요구되는 정보

- ConnectionPool

connection pool 기능을 이용할 때만 아래의 정보가 추가로 입력된다.

- MinPoolSize : pool의 초기 크기
- MaxPoolSize : pool의 최대 크기
- IncreasingStep : pool에 여분이 없을 때 최대 pool 크기까지 connection pooling 수를 증가시키는데 그 때 한 번에 증가되는 connection수
- ResizingPeriod
분단위로 입력하여야 하며, 해당 시간을 주기로 사용되지 않는 connection들을 MinPoolSize까지 줄인다.
- WaitForFreeConnection

만약 최대 크기까지 pool이 증가한 이 후로도 요청이 계속들어 오면 바로 connection을 새로 만들어 주고 이렇게 생성된 connection은 pool로 귀환되지는 않도록 하거나

일단 지정된 시간만큼 계속 pool을 검사하다가 그때까지도 connection을 얻을 수 없으면 exception을 던지는 방법이 있다.

2)번 방법을 이용할 경우 아래의 WaitingTime정보를 추가한다.

<WaitingTime> : connection을 얻기 위해 대기하는 시간. mSec 단위로 입력함

먼저, JeusMain.xml에서의 데이터소스 구성에 대해 알아본다.

각 벤더마다 설정이 다르므로 주의해서 설정을 해야 한다. 설정이 끝나면 반드시 바인드를 해서 연결이 맺어지는지 확인을 해야 한다.

9. Vendor Specific DataSource Property

9.1 Oracle

- Oracle에서 제공하는 JDBC 드라이버는 스펙에 나오는 모든 데이터소스 프라퍼티를 준수하지 않는다. 아래 테이블은 Oracle에서 추가적으로 제공하는 프라퍼티를 정리한 것이다.

이름	형 태(Type)	설 명
driverType	String	사용하고자 하는 Oracle JDBC driver 의 카테고리 설정 가능한 값 : oci8, thin, kprb
url	String	데이터베이스 연결 URL. 용도는 Oracle Migration 시 편리하게 사용하기 위해 만들어졌다. 이 프라퍼티가 설정되면, tnsEntry 및 driverType, portNumber, networkProtocol, serverName, databaseName 값은 무시된다.
tnsEntry	String	TNS entry name – 오직 OCI 드라이버와 관련되며, Oracle 클라이언트에는 환경변수 TNS_ADMIN 이 설정되어 있어야 한다.

표2-3 Oracle specific DataSource Property

Note : 제우스에서는 추가적 프라퍼티 중에서 driverType을 지원한다.

Oracle*s JDBC Thin 드라이버는 TYPE 4 JDBC 드라이버로, Java Socket 을 사용하여 오라클 데이터베이스에 직접 접속할 수 있으며 client 장비에 오라클 제품을 필요로 하지 않는다.thin 드라이버는 oracle 홈페이지에서 download 받을 수 있으며 JEUS_HOME/lib/datasource에 위치 시켜놓으면 된다.

다음으로는 Oracle*s JDBC oci드라이버에 대한 설명이다.

Oracle의JDBC OCI 드라이버는 TYPE 2 JDBC 드라이버로,오라클 데이터 베이스와 상호 연동하기 위해 OCI(Oracle Call Interface)를 사용하여

JDBC 인터페이스 구현을 제공한다.

설치는 oracle 홈페이지에서 download 받을 수 있으며 설치 위치는 다음과 같다. JEUS_HOME/lib/system에 위치 시켜 놓으면 된다.

Oracle에 관한 데이터소스 설정을 살펴보면 아래와 같다.

```
<DataSource>
<Database>
<Vendor>oracle</Vendor>
<ExportName>test_or</ExportName>
// 오라클 데이터소스의 JNDI 이름
<DatabaseName>ORCL</DatabaseName>
// 데이터베이스 이름. SID값
<DataSourceName>oracle.jdbc.pool.OracleConnectionPoolDataSource</DataSourceName>
// Oracle의 연결 풀 데이터소스 이름
    <Description />          // 생략 가능
    <NetworkProtocol />      // 기본값 사용. 기본 값은 TCP
<Password>tiger</Password>
<PortNumber>1521</PortNumber>
<ServerName>pollux5</ServerName>
<User>scott</User>
<DriverType>thin</DriverType> //driver Type을oci8로 하면 ociJDBC-Driver로 설정
    되어진다.
<ConnectionPool>
<MinPoolSize>6</MinPoolSize> // 초기 pool 크기는6
<MaxPoolSize>10</MaxPoolSize> // 최대 10
<IncreasingStep>2</IncreasingStep> // pool의 크기 증가가 필요하면2개씩 증가시킴
<ResizingPeriod>3</ResizingPeriod> // 3분 마다 사용 안되는 connection 없앰
<WaitForFreeConnection>
<WaitingTime>10000</WaitingTime> // 10 초간 대기하며 connection 얻기를 시도함
</WaitForFreeConnection>
</ConnectionPool>
</Database>
</DataSource>
```

9.2 Sybase

사이베이스(Sybase)의 경우도 Oracle과 마찬가지로 jConnect라는 자신의 JDBC 드라이버를 가지고 있다. 데이터소스 프라퍼티 중roleName을 지원하지 않는다. 또한 jConnect는javax.naming.spi.ObjectFactory 인터페이스를 직접 구현함으로써 Naming Server Entry의 속성(attribute)으로부터 데이터소스 객체를 얻을 수 있다. 예를 들면 LDAP 프로토콜을 사용하여 데이터소스 객체를 얻을 수 있는 것이다. 또한, 데이터소스 객체 프라퍼티 중 Description은 반드시 기입해야 한다. JDBC 드라이버 jConnect는Sybase Server와 통신하기 위해 Sybase 데이터베이스 서버의 Native 통신 프로토콜인 TDS(Tabular Data Stream)을 사용한다. NetworkProtocol은 생략이

가능하나, 값을 설정할 경우에는 반드시 Tds를 사용한다. 대소문자에 주의하도록 한다.

sybase에서 데이터베이스에 접속하기 위해서는 “isql” 프로그램을 사용한다.

프로그램 설치와 환경설정이 끝났으면 “isql -U -P” 명령을 이용하여 데이터베이스에 접속할 수 있다.

예를 들면 `$>isql -User -Ppassword`

데이터베이스에 접속을 하였으면 isql 에서 질의를 서버에 보내기 위해서는 go를 사용한다. 현재의 환경 설정을 확인하려면 sp_configure를 사용한다. 질의를 마친 후isql에서 빠져나오기 위해서는 quit나exit를 사용하여 빠져나올 수 있으며 서버를 중지시키려면 shutdown를 하고 빠져나오면 된다.

```
<DataSource>
<Database>
<Vendor>sybase</Vendor>
<ExportName>test_sy</ExportName>
<DatabaseName>master</DatabaseName>
// DB 담당자에게 물어서 설정한다.
<DataSourceName>com.sybase.jdbc2.jdbc.SybConnectionPoolDataSource</DataSourceName>
// 사이베이스 연결 풀 데이터소스 이름
<Description>test11</Description>
<NetworkProtocol />
<Password>xlaortm</Password>
<PortNumber>4100</PortNumber>
<ServerName>pollux5</ServerName>
<User>sa</User>
<DriverType />
<ConnectionPool>
<MinPoolSize>6</MinPoolSize>
<MaxPoolSize>10</MaxPoolSize>
<IncreasingStep>2</IncreasingStep>
<ResizingPeriod>3</ResizingPeriod>
    이 세팅에서는 <WaitForFreeConnection> 이 없으므로 최대 pool 크기 상태에서
    추가적인 connection 요청이 있으면 새로 connection을 생성하여 한번 사용
    하고 없애는 방식(1)을 취하게 된다.
</ConnectionPool>
</Database>
</DataSource>
```

9.3 db2

DB2는 버전5에서부터 자바 응용프로그램의 JDBC지원을 하게 되었다.

JDBC는 자바 애플리케이션이 데이터베이스를 액세스하기 위한 API(

Application Programming Interface)로서 동적(Dynamic) SQL의 형태로 데이터베이스를 액세스한다. 기존의 ODBC(Open DataBase Connectivity)의

연결성의 장점에 자바 기술을 접목한 형태로 JDBC-ODBC Bridge 형태가 아닌 JNI(자바 Native Interface)를 사용한다.

DB2 UDB(Universal Database)에서는 App Driver (JDBC type2) 또는 net Driver (JDBC type3)의 2가지 형태의 JDBC Driver를 제공하며, 원격으로 DB2 server에 접근 할시 Program을 작성시엔 net Driver를 사용하여야 한다.

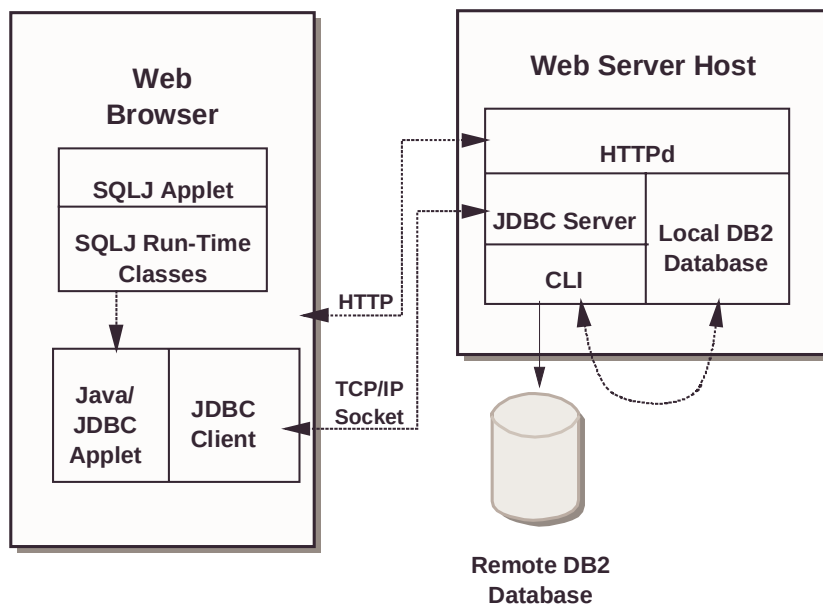


그림2-1 DB2 JDBC Driver (net Driver) 구성도

Net Driver의 JDBC Driver는 위와 같은 형태의 아키텍처를 가지므로, DB2 Server (또는 Web Server)에서 별도의 process로 실행해줘야 한다.

9.3.1 DriverManager 이용방법

DriverManager 을 이용하는 방법에는 2 가지 방법이 있다.

9.3.1.1 UDB DB2 App Driver(type2) 이용방법

```

<container>
.
.
.
<DBConnectionPool ConnectionPoolID="nonSharedoraclePool"
    ConnectionPoolType="non-shared"
    ThreadPort="8088"
    ConnectionURL="jdbc:db2://serverIP:6788/DBalias"//port 6788

```

```

        DriverClassName="COM.ibm.db2.jdbc.app.DB2Driver"
        ConnectionArguments="user=db2inst1;password=jeus"
        CloseLongActiveConnection="true"
        MaxActiveTimeSecs="60">
    </DBConnectionPool>
    . . .
</container>

```

9.3.1.2 UDB DB2 Net Driver(type4) 이용방법

```

<container>
    . . .
    <DBConnectionPool ConnectionPoolID="nonSharedoraclePool"
        ConnectionPoolType="non-shared"
        ThreadPort="8088"
        ConnectionURL="jdbc:db2://serverIP:6788/DBalias"//port 6788
        DriverClassName="COM.ibm.db2.jdbc.net.DB2Driver"
        ConnectionArguments="user=db2inst1;password=jeus"
        CloseLongActiveConnection="true"
        MaxActiveTimeSecs="60">
    </DBConnectionPool>
    . . .
</container>

```

Command prompt상에서 다음과 같이 입력한다.

“db2jstrt port번호”

ex) db2jstrt 6788

해당 port 번호는 client에서 실행될 같은 port 번호로 setting되어야 하며, 해당 process의 실행 여부는 “db2jd port번호” process를 확인한다.

db2의 경우도 Oracle,Sybase와 마찬가지로 db2java라는 자신의 JDBC 드라이버를 가지고 있다. DatabaseName은db2에서 만든 DBalias명을 기입해야 하며,User와Password만 등록하면 된다.

9.3.2 JNDI 이용방법

```

<Database>
    <Vendor>db2</Vendor>
    <ExportName>db2xa</ExportName>
    <DatabaseName>SAMPLE</DatabaseName>//DBalias
    <DataSourceClassName>COM.ibm.db2.jdbc.
        DB2ConnectionPoolDataSource</DataSourceClassName> //db2연결 풀 데이터소스 이름
    <DataSourceName/>
    <ServiceName/>
    <Description/>
    <NetworkProtocol/>

```

```

<Password>jeus</Password>//db2패스워드
<PortNumber>0</PortNumber>
<ServerName/>
<User>db2inst1</User>//db2 user
<DataSourceType>ConnectionPoolDataSource</DataSourceType>
<ConnectionPool>
<MinPoolSize>2</MinPoolSize>
<MaxPoolSize>4</MaxPoolSize>
<IncreasingStep>1</IncreasingStep>
<ResizingPeriod>600000</ResizingPeriod>
<OperationTimeout>30000</OperationTimeout>
<WaitForFreeConnection>
<WaitingTime>10000</WaitingTime>
</WaitForFreeConnection>
</ConnectionPool>
</Database>

```

9.4 MS-SQL

Microsoft 에서 만든 SQL을 지원하는 Database System 이고,MS Windows 에만 설치가 가능하면 서 접속을 시도할 경우, ODBC를 통해서만 접근이 가능하다.

JEUS에서 접근할 경우, System의ODBC 설정에서 SQL Server와 접속할 수 있도록 설정해 놓은 다 음에 DataSource를 만들어야 한다.

```

<Database>
<Vendor>jeus_mssql</Vendor>
<ExportName>mssql</ExportName>
<DatabaseName/>
<DataSourceClassName>jeus.jdbc.driver.mssql.MSSQLConnectionPoolDataSource<
/DataSourceClassName>
<DataSourceName/>
<ServiceName/>
<Description/>
<NetworkProtocol/>
<Password/>
<PortNumber>-1</PortNumber>//MS-SQL은portNumber를 지원하지 않는다.
<ServerName>mssql</ServerName>
<User>sa</User>
<DataSourceType>DataSource</DataSourceType>
</Database>

```

10. 트랜잭션을 이용한 무상태 세션 빈 예제

제우스 서버는 클라이언트에서의 트랜잭션을 지원하지 않는다. 즉 클라이언 트 응용 프로그램내에서 트랜잭션에 관한 객체를 얻을 수 없다. 또한, 스 펙에서 클라이언트에서의 트랜잭션 수행을 권고하지 않는다. 여기서는 스 펙에서 권고하는 서버내에서 XA트랜잭션을 위해 빈에서 어떻게 트랜잭션

을 얻는가에 대해서 언급한다. 그리고, 이를 제우스 서버에서 어떻게 설정하는 가를 설명한다. 빈의 트랜잭션 수행타입은 Bean Managed 이다. 빈은 트랜잭션을 지원하는 연결 풀을 이용해서 데이터베이스 테이블의 row를 갱신(update) 한다.

빈을 수행하기 위해서 아래의 과정을 수행한다.

- XADatasource를 등록한다.
- 관리 툴을 사용하여 데이터소스를 등록한다. 툴을 통하여 만들어진 JeusMain.xml의 일부분이다.

10.1 Oracle

```
<Database>
<Vendor>oracle</Vendor>
<ExportName>jdbc/xa/CoffeesDB</ExportName>
<DatabaseName>ORA816</DatabaseName>
<DataSourceName>oracle.jdbc.xa.client.OracleXADataSource</DataSourceName>
<Description />
<NetworkProtocol />
<Password>tiger</Password>
<PortNumber>1521</PortNumber>
<ServerName>stand</ServerName>
<User>scott</User>
<DriverType>thin</DriverType>
<DataSourceType>XADataSource</DataSourceType>
<ConnectionPool>
<MinPoolSize>2</MinPoolSize>
<MaxPoolSize>20</MaxPoolSize>
<IncreasingStep>2</IncreasingStep>
<ResizingPeriod>10</ResizingPeriod>
<WaitForFreeConnection>
<WaitingTime>30000</WaitingTime>
</WaitForFreeConnection>
</ConnectionPool>
</Database>
```

10.2 Sybase

```
<Database>
<Vendor>sybase</Vendor>
<ExportName>XAsybase</ExportName>
<DatabaseName>testdb</DatabaseName>
<DataSourceClassName>com.sybase.jdbc2.jdbc.SybXADataSource</DataSourceClassName>
<DataSourceName>com.sybase.jdbc2.jdbc.SybXADataSource</DataSourceName>
<ServiceName/>
<Description>sybase</Description>
<NetworkProtocol>Tds</NetworkProtocol>
```

<Password>sybase</Password>//분산트랜잭션을 위해서는 Password는 반드시 넣어 주어야 한다.

```
<PortNumber>4100</PortNumber>
<ServerName>sybaseserver</ServerName>
<User>sa</User>
<DataSourceType>XADataSource</DataSourceType>
<ConnectionPool>
  <MinPoolSize>10</MinPoolSize>
  <MaxPoolSize>50</MaxPoolSize>
  <IncreasingStep>5</IncreasingStep>
  <ResizingPeriod>10</ResizingPeriod>
  <OperationTimeout>30000</OperationTimeout>
  <WaitForFreeConnection>
    <WaitingTime>10000</WaitingTime>
  </WaitForFreeConnection>
</ConnectionPool>
</Database>
```

10.3 db2

```
<Database>
  <Vendor>db2</Vendor>
  <ExportName>db2xa</ExportName>
  <DatabaseName>SAMPLE</DatabaseName>
  <DataSourceClassName>COM.ibm.db2.jdbc.DB2XADataSource</DataSourceClassName>
  <DataSourceName/>
  <ServiceName/>
  <Description/>
  <NetworkProtocol/>
  <Password>jeus</Password>
  <PortNumber>0</PortNumber>
  <ServerName/>
  <User>db2inst1</User>
  <DataSourceType>XADataSource</DataSourceType>
  <ConnectionPool>
    <MinPoolSize>2</MinPoolSize>
    <MaxPoolSize>4</MaxPoolSize>
    <IncreasingStep>1</IncreasingStep>
    <ResizingPeriod>600000</ResizingPeriod>
    <OperationTimeout>30000</OperationTimeout>
    <WaitForFreeConnection>
      <WaitingTime>10000</WaitingTime>
    </WaitForFreeConnection>
  </ConnectionPool>
</Database>
```

10.4 MS-SQL

MS-SQL은Local 트랜잭션,XA 트랜잭션을 제공하지 않는다.

- JeusMain.xml을 구성한 후, 엔진을 기동한다.

- Bean-Managed 빈에서 트랜잭션을 수행하기 위해서는 UserTransaction 객체를 얻어야 한다. 이 객체를 얻는 방법은 두 가지이다. EJBContext에서 얻거나 JNDI를 통하여 얻는다.
- 먼저, javax.xa.transaction.* Package를import한다.
- Session Context sc에서 UserTransaction을 얻는다.
- UserTransaction ut = sc.getUserTransaction()
- JNDI를 이용하는 방법은 다음과 같다.
- Context ctx = InitialContext();
- UserTransaction ut = ctx.lookup("java:comp/UserTransaction");
- 어떤 방법을 사용하든 상관없다. 여기서는 두 번째 방법을 사용한다.
- UserTransaction의begin() 과end() 함수를 통하여 transaction scope를 설정한다. 이 사항은 오직 트랜잭션 타입이 Bean-Managed 일때만 적용된다.
- 그리고, 트랜잭션 수행 중Exception이 발생할 경우, rollback 함수를 호출하여 트랜잭션을 rollback한다.

나머지 코드는 일반적인 무상태 세션 빈 코드와 동일하다. 그리고, EJB Module을 생성할 때, 트랜잭션 타입을 Bean으로 설정한다. 제우스의 EJB Module Descriptor는ejb-jar-modulename.xml이다. 배치를 할 때 사용하는 Descriptor는jeus-ejb-dd_modulename.xml이다. 툴을 이용해서 EJB Module을 생성할 때ejb-jar_Coffees.xml을 사용하고, 배치(deploy)를 할 때ejb-jar-dd_Coffees.xml을 사용한다. 예제 코드는 부록을 참고한다.

11. 툴을 통한 연결 풀 구성

관리자 툴에서 연결 풀을 생성하기 위해 시JeusMain.xml에 미리 구성을 할 경우, 제우스 노드 관리자가 시작되면서 JNDI에 데이터 소스를 등록하게 된다.

- 메뉴 바에서 Configuration --> JeusSystem이나 를 선택한다.
- JEUS System Config file Dialog창이 나타나면, 구성 파일을 일괄적으

로 정리할 장소 - 여기서는 제우스 관리 디렉토리 -로 이동한 후, JeusMain.xml을 생성한다

- JeusMain Descriptor Dialog 창에서 DataSource Section에서 Add 버튼을 눌러 DataSource를 추가한다.

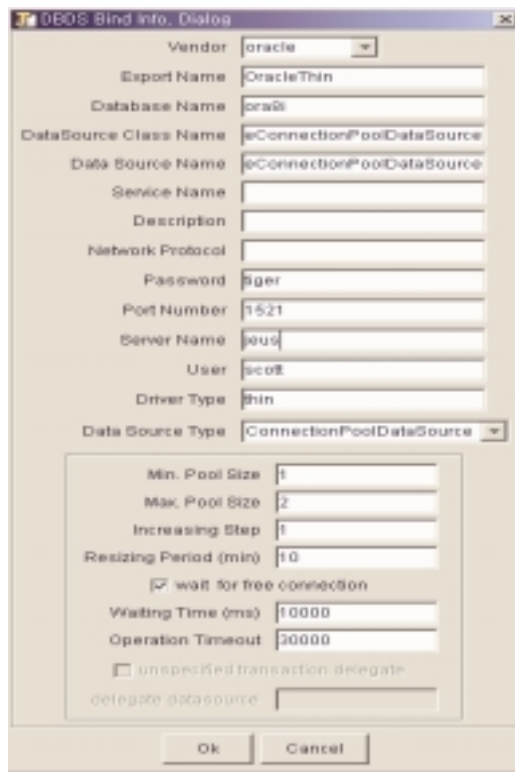


그림2-2데이터소스 추가

위 의 경우는 Oracle의 경우를 예를 든 것이다.

Note : 툴을 통한 JeusMain.xml의 구성은 제우스 툴 매뉴얼을 참고한다.

11.1데이터 소스 명령

JNDI를 통해 데이터 소스를 사용할 수 있도록 등록(bind) 하는 명령이다.

데이터 소스는 노드 단위로 등록(bind) 할 수 있으므로 우선 시스템 트리창에서 데이터 소스를 등록(bind)하고자 하는 노드를 선택하고 System Control메뉴의 Bind Data Source 항목을 선택하거나 아이콘바의 데이터소

스() 버튼을 누르면 위와 동일한 등록(bind) 정보를 입력하는 대화상자가 나타난다.

항목별 값을 입력하고 Ok 버튼을 선택하면 해당 노드의 제우스 서버에 명령이 전달되며 결과표시 대화상자에 처리 결과가 나타난다.

III . dbpooladmin을 이용한 connection pool 상태 관리

connection pool수행중 Bean deploy시console에서 Bean deploy에 대한 정보를 확인 및connection pool에 대한 동작를 control할 수 있다.

이는 client에서 Bean을 호출시 console및Administration Tool에서 connection pool를 관리 할 수 있다.

dbpooladmin에서 제공하는 명령어들은 다음과 같이 나눌 수 있다.

- engine container 대한 connection pool에 대한 정보를알려주는 명령어 (info)
- connection pool 동작을 조절하는 명령어들 (enable, disable, shrink, resync)
- 그 외의 명령어(help)

위의 프로그램은 JEUS_HOME/bin아래에 있으며 dbpooladmin을 실행시키는 방법은 다음과 같다.

```
[ConnectionPoolController] dbpooladmin [ | -verbose] [engine container name] [ | node name]
```

예를 들면 dbpooladmin -verbose jeus_container1 jeus라 기입하든지

아니면 dbpooladmin jeus_container1 jeus기입해도 무방하다.

위의 과정을 수행하고 나면 다음과 같은 화면이 나온다.

```
[ErrorMsgManager] Message Manager is initialized
[ConnectionPoolController] Jeus 3.0 DB Connection Pool Controller
[ConnectionPoolController] Login :
>
[ConnectionPoolController] Password :
>
```

userID와Password를 기입하면 된다. 만일 UserID와Password가 틀린경우는 다음과 같은 화면이 출력된다.

```
[ConnectionPoolController] a problem in checking security for UserID
jeus.security.JSException
at jeus.security.JeusSecurityManager.login(JeusSecurityManager.java:261)
at jeus.jdbc.connectionpool.ConnectionPoolController.main(ConnectionPoolController.java:305)
[JeusSecurityManager] User Not Authenticated (Invalid Password) :jeus
.security. JSException
```

정상적으로 UserID와Password를 기입했을때의 화면이다.

```
jeus_container1>
```

우선 dbpooladmin이 제공하는 명령어들의 종류와 사용방법을 알기 위해서 help를 실행시키면 다음과 같은 화면이 출력이 된다.

```
jeus_container1>help
dbpooladmin command
• info : gather informations for connection pools in the engine
container
• enable [connection pool name] : enable the connection pool
ex> enable mydbpool -> enable mydbpool
• disable [connection pool name] : disable the connection pool
ex> disable mydbpool -> disable mydbpool
• shrink [connection pool name] : shrink the size of the connection pool
by clo
sing idle connections
ex> shrink mydbpool -> shrink mydbpool
• resync [connection pool name] : resync global transaction statue for
the conn
ection pool
ex> resync mydbpool -> recover the status of the global transaction done
wit
h mydbpool
```

위의 dbpooladmin을 종료하고자 할때는 exit 명령어를 수행하면된다.

1. dbpooladmin 에서 connectionpool 정보를 알려주는 명령어

1.1 console인 경우

우선 Bean을deployment Tool및console에서 deploy를 먼저 실행한다.

예를 들어 예제에 있는 bmacount를deploy를 하기로 하자.

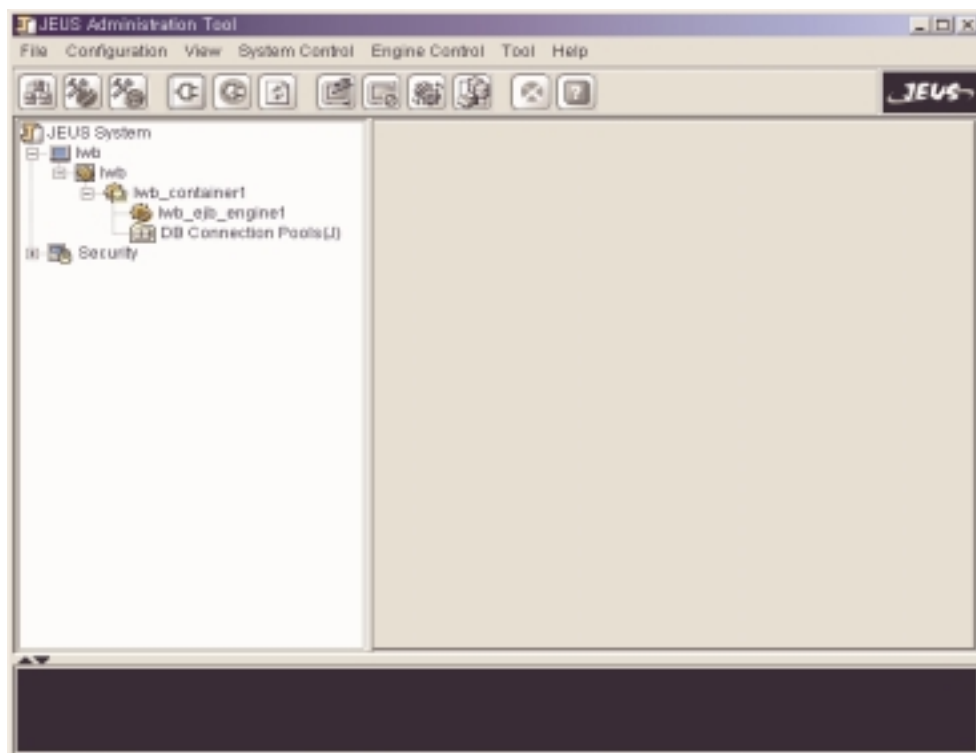
Deploy에 대한 자세한 내용은 *deployment Tool*을 참고 하기 바란다.

Deploy가 성공적으로 되었을 경우 console에서 다음과 같이 명령어를 입력하고 확인한다.

```
ex)jeus_container1>info
=====
connection pool name : dataSource1
min size : 1
max size : 2
current size : 1
idle size : 1
wating : true
working : true
=====
```

1.2 admininstration Tool인 경우

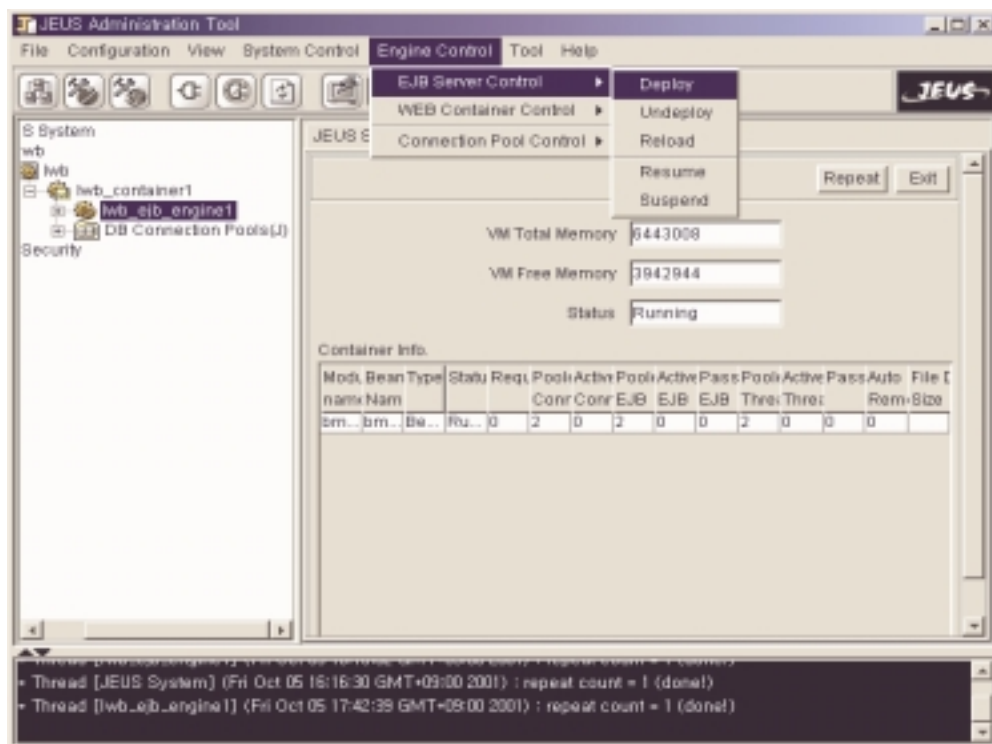
administration Tool에서 deploy를 실행 한다.



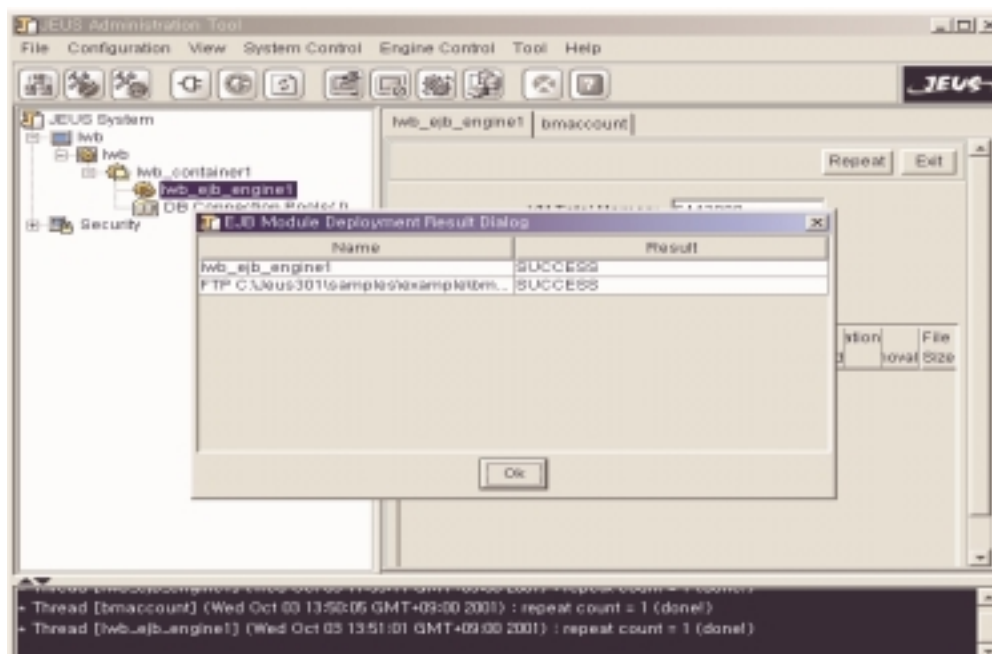
<그림 3-1>

아래 그림처럼 Engine control➡ EJB Server Control➡deploy를 시켜야

된다.

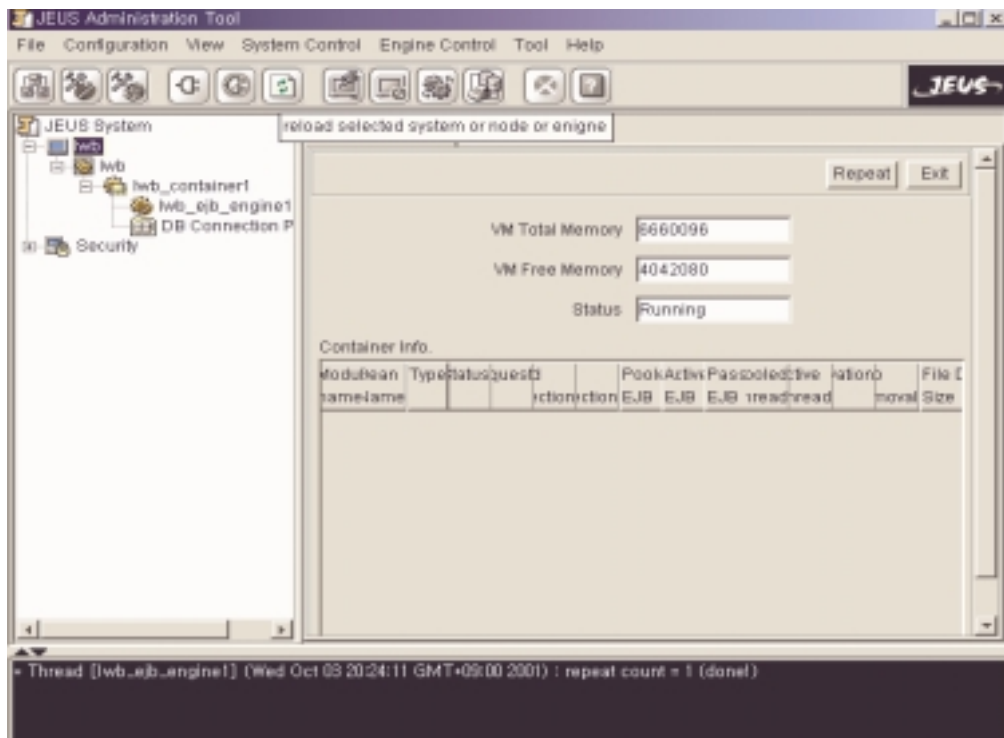


<그림 3-2>



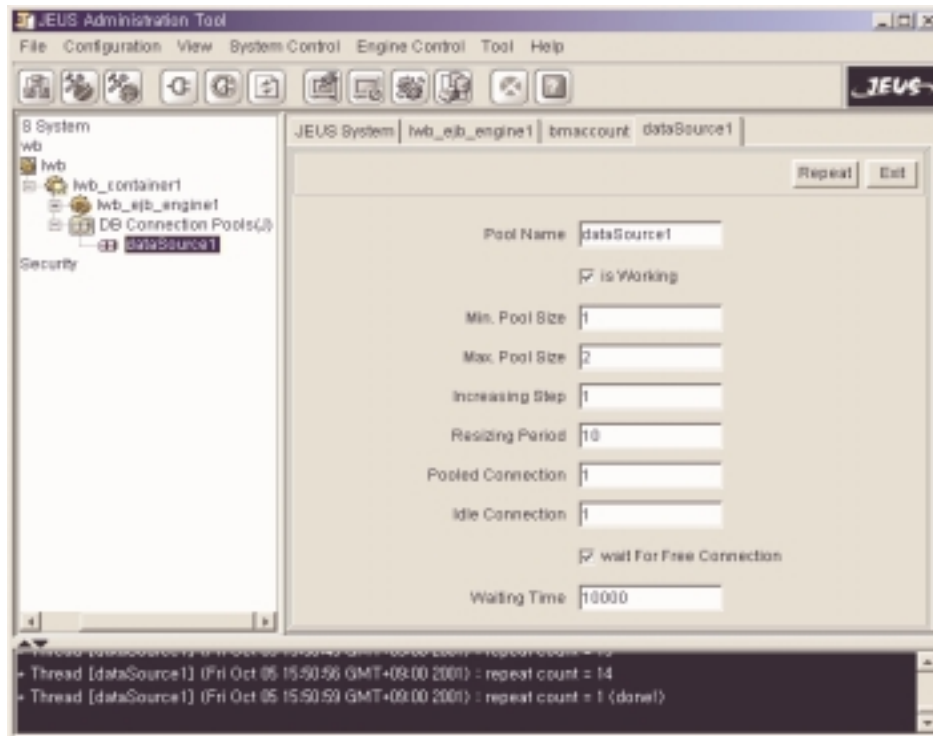
<그림 3-3>

위의 화면은 deploy가 성공했다는 메시지 화면이다.



<그림 3-4>

deploy가 끝나면 reload를 다시해야만 DB Connection pool에 등록이 된다.



<그림 3-5>

위의 화면에서 DBConnection Pools dataSource1라는것은 bmacount bean에서 Look up 하는 DataSource 명이 적혀 있는 것을 확인할 수 있을 것이다.

2. dbpooladmin 에서 connectionpool 상태를 control 하는 명령어

2.1 console

- disable

console에 다음과 같이 기입하기전 deploy가 되어 있어야 한다.

```
jeus_container1>disable DataSource명  
ex) jeus_container1>disable dataSource1
```

client에서 Bean를 호출해보자 connectionpool이 생성이 안됨을 확인 할 수 있을 것이다.

■ enable

```
jeus_container1>enable DataSource명  
ex) jeus_container1>enable dataSource1
```

client에서 Bean를 호출해보자 connectionpool이 생성이 되었음을 확인 할 수 있을 것이다.

■ Shrink

```
jeus_container1>shrink DataSource명  
ex) jeus_container1>shrink dataSource1
```

connection pool 사용후 idle 된 connection pool들을 없애는 명령어이다. 이는 자원의 절약을 위함이다.

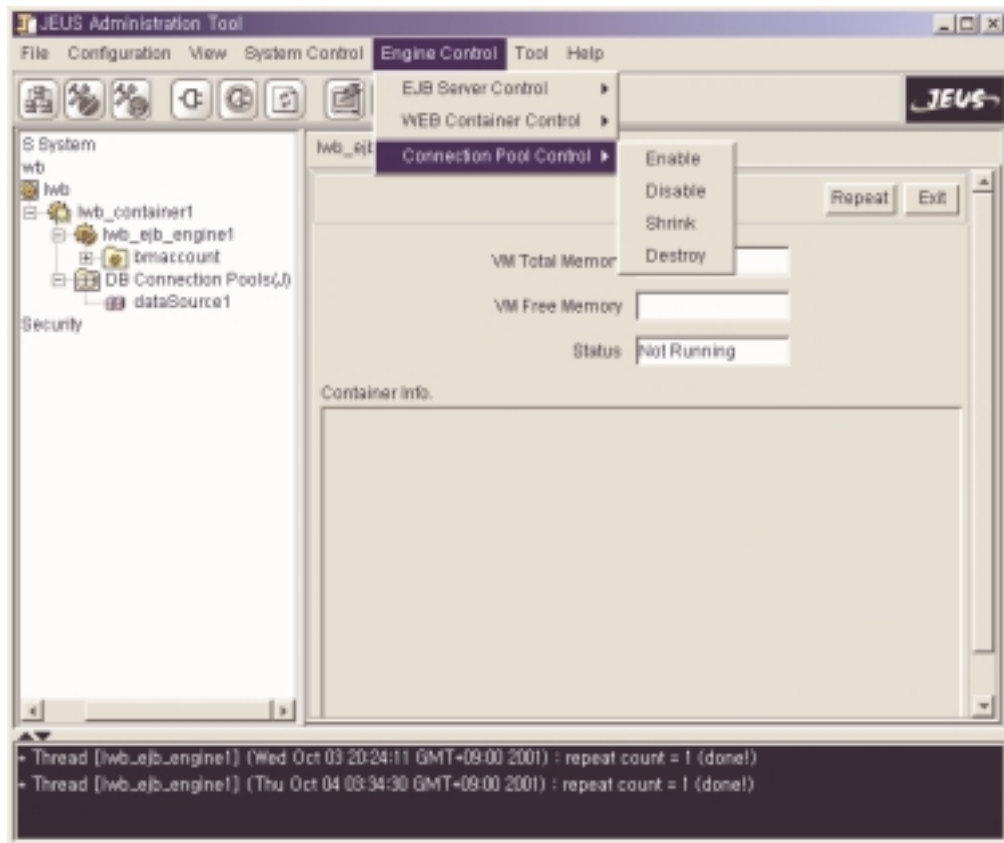
■ resync

```
jeus_container1>resync DataSource명  
ex) jeus_container1>resync dataSource1
```

XA transaction 과정 중 db에 문제가 있을 경우 XA transaction은 commit를 수행하지 못하고 rollback를 수행하게 되는데, 이후 다시 XA transaction을 수행시 resync를 사용하게 되면 기존의 XA transaction 과정의 log를 찾아 다시 수행하게하는 명령어이다.

2.2 Administration Tool

Tool에서는 다음과 같이 실행한다.



<그림 3-6>

위의 그림처럼 Engine Control→Connection Pool Control를 클릭하면 위의 화면처럼 선택할 수 있다.

기능은 위의 console과 동일하다.

IV .Sample Source를 통한 프로그램 구현

1. ConnectionPoolDataSource에 의한 Stateful session Bean

1.1 Oracle

```
test.sql 생성
create table test
(Id varchar2(20),
Passwd varchar(20));
```

```
//Remote Interface 부분
package oracle;//패키지명
import java.rmi.*;
import java.sql.*;
import javax.sql.*;
import javax.ejb.*;
```

```
public interface OracleRemote extends EJBObject {
public void query()throws RemoteException, SQLException;//query()비즈니스 메
소드
}
```

```
//HomeInterface 부분
package oracle;//패키지명
import java.rmi.*;
import javax.ejb.*;

public interface OracleHome extends javax.ejb.EJBHome {
public OracleRemote create() throws RemoteException, CreateException;
}
```

```
//Bean class 부분
package oracle;//패키지명
import java.sql.*;
import javax.sql.*;
import javax.naming.*;
import javax.ejb.*;

public class OracleBean implements SessionBean {
public void OracleBean() {}
private Connection con = null;
private ResultSet rs=null;
private String test1="";//test의 필드1
private String test2="";//test의 필드2
```

```

private SessionContext sc = null;
private Context ctx = null;
private DataSource ds =null;

public String ejbCreate() throws CreateException {
try {
ctx = new InitialContext();
System.out.println("[#####IntialContext() sucess!!]");
ds = (DataSource) ctx.lookup("Oracle");//JeusMain.xml에서
        ExportName0|oracle를look up 하는 부분
System.out.println("[Datasource connection suecss !!]");
}catch (Exception e)
        {
e.printStackTrace();
} return "sucess";
    }

public void query() throws SQLException {
try {
con = ds.getConnection();
Statement stmt = con.createStatement();
rs = stmt.executeQuery("select * from test");
while (rs.next()) {
test1 = rs.getString(1);
test2 = rs.getString(2);
System.out.println("*****"+test1);
System.out.println("*****"+test2);
        }

    } catch (Exception e)
        {
e.printStackTrace();
        }
finally
    {
        if( rs!=null) rs.close();
        if (con != null) con.close();
    }

    }

public void setSessionContext(SessionContext sc) {
this.sc = sc;
    }

public void ejbRemove() {}
public void ejbPassivate() {}
public void ejbActivate() {}
}

//Client 부분
package oracle;

```

```

import java.sql.*;
import javax.sql.*;
import javax.naming.*;
import javax.ejb.*;
import javax.rmi.*;
import oracle.*;

class OracleClient {
public static void main(String[] args) {
try {
Context ctx = new InitialContext();
Object obj = ctx.lookup("test");//Bean class와의 look up 부분
        OracleHome home
=(OracleHome)PortableRemoteObject.narrow(obj, OracleHome.class);
OracleRemote ora = home.create();
ora.query();//메소드 호출

    }
catch (Exception e)
    {
e.printStackTrace();
    }
}
}

```

```

//JeusMain.xml
<DataSource>
<Database>
<Vendor>oracle</Vendor>
<ExportName>Oracle</ExportName>
<DatabaseName>ora8i</DatabaseName>
<DataSourceClassName>oracle.jdbc.pool.OracleConnectionPoolDataSource</Data
SourceClassName>
<DataSourceName>oracle.jdbc.pool.OracleConnectionPoolDataSource</DataSourc
eName>
<ServiceName/>
<Description/>
<NetworkProtocol/>
<Password>tiger</Password>
<PortNumber>1521</PortNumber>
<ServerName>jeus</ServerName>
<User>scott</User>
<DriverType>thin</DriverType>
<DataSourceType>ConnectionPoolDataSource</DataSourceType>
<ConnectionPool>
<MinPoolSize>1</MinPoolSize>
<MaxPoolSize>2</MaxPoolSize>
<IncreasingStep>1</IncreasingStep>
<ResizingPeriod>10</ResizingPeriod>
<OperationTimeout>30000</OperationTimeout>
<WaitForFreeConnection>
<WaitingTime>10000</WaitingTime>
</WaitForFreeConnection>

```

```

</ConnectionPool>
</Database>

```

다음은 OracleClient.java를 실행 하면 서버에 출력되는 화면이다.

```

[2001.09.26 00:31:48] [JNDI.CONTEXT] REBIND <test>
[2001.09.26 00:31:48] [JMServer] REBIND <test> (SUCCESS) Sent.
[2001.09.26 00:31:48] [Container] test is now bound in Naming Server
[2001.09.26 00:31:48] [JMSLocal_] <recv> CACHE DROP <test>
[2001.09.26 00:31:48] [JMSLocal_] <recv> CACHE DROP <test>
[2001.09.26 00:31:48] [JMSLocal_] <recv> CACHE DROP <test>
[2001.09.26 00:31:48] [JMSLocal_] <recv> CACHE DROP <test>
[2001.09.26 00:31:48] [EJBServer] Module Deployment Process Successful for data
oracle
[2001.09.26 00:31:48] -----
[2001.09.26 00:32:00] [JMServer] <ACCEPT> Local server : localhost(127.0.0.1:27
92)--<9920>
[2001.09.26 00:32:00] [JMSSecurityManager] Run as guest
[2001.09.26 00:32:00] [JNDI.CONTEXT] <success!!!>
[2001.09.26 00:32:00] [JNDI.CONTEXT] LOOKUP <Oracle>
[2001.09.26 00:32:00] [Data source] connection success !!!
[2001.09.26 00:32:00] [TimerServer] a coordinator created <61.77.153.61:0:704022202
1>
[2001.09.26 00:32:00] [Coordinator] <61.77.153.61:0:704022202:1> : a synchroniz
ation registered
=====test1
=====test2
[2001.09.26 00:32:00] [Terminator] <61.77.153.61:0:704022202:1> : commit starts
[2001.09.26 00:32:00] [Coordinator] <61.77.153.61:0:704022202:1> : beforeComple
tion
[2001.09.26 00:32:00] [Coordinator] <61.77.153.61:0:704022202:1> : prepare resu
lt<2>
[2001.09.26 00:32:00] [Terminator] global decision of <61.77.153.61:0:704022202
1> is 2
[2001.09.26 00:32:00] [Coordinator] <61.77.153.61:0:704022202:1> : afterComple
tion
[2001.09.26 00:32:00] [Coordinator] <61.77.153.61:0:704022202:1> : destroyed

```

그림4-1 oracle ConnectionPoolDataSource 실행 출력 화면

2. LocalXADataSource에 의한 Stateful session

Bean

2.1 db2

```

test1.sql 생성
create table test1
(Id varchar2(20),
Passwd varchar(20));

```

```

Test2.sql 생성
Create table test2
(Id varchar2(20),
Passwd varchar(20));

```

```

//RemoteInterface
package db2;
import java.rmi.*;
import java.sql.*;
import javax.sql.*;
import javax.ejb.*;

```

```

public interface Db2Remote extends EJBObject {
public void query()throws RemoteException, SQLException; //query()비즈니스  
메소드
}

```

//HomeInterface

```

package db2;
import java.rmi.*;
import javax.ejb.*;

```

```

public interface Db2Home extends javax.ejb.EJBHome {
public Db2Remote create() throws RemoteException, CreateException;
}

```

//Bean class

```

package db2;
import java.sql.*;
import javax.sql.*;
import javax.naming.*;
import javax.ejb.*;
import javax.transaction.*;

```

```

public class DbBean implements SessionBean {
public void DbBean () {}
private Connection con = null;
private ResultSet rs=null;
private String test1="";
private String test2="";
private PreparedStatement pstmt=null;
private Statement stmt=null;
private SessionContext sc = null;
private Context ctx = null;
private DataSource ds =null;
private UserTransaction ut = null;

```

```

public String ejbCreate() throws CreateException {
try {
ctx = new InitialContext();
System.out.println("[InitialContext...]");
ut = (UserTransaction)
ctx.lookup("java:comp/UserTransaction");//Bean-Managed Transaction 관련  
JTS참고
System.out.println("[db2 Data Source connect...]");

```

```

ds = (DataSource) ctx.lookup("db2local");//JeusMain.xml에서 ExportNameO
oracle를look up 하는 부분
System.out.println("[db2 Data Source sucess !!]");
}catch (Exception e)
{

e.printStackTrace();
} return "sucess";
}

public void query() throws SQLException {
try {
ut.begin();//Transaction 시작
con = ds.getConnection();
Statement stmt = con.createStatement();
rs = stmt.executeQuery("select * from test1");

while (rs.next()) {
test1 = rs.getString(1);
test2 = rs.getString(2);
System.out.println("*****"+test1);
System.out.println("*****"+test2);
}

PreparedStatement pstmt = con.prepareStatement("insert into test2 values
( ?, ?)");
pstmt.setString(1,test1);
pstmt.setString(2,test2);
pstmt.executeUpdate();

ut.commit();//Transaction완료
} catch (Exception e)
{
e.printStackTrace();
try {
ut.rollback();//Transaction 실패
} catch (Exception ee)
{
ee.printStackTrace();
}
}finally
{
if (pstmt != null) pstmt.close();
if( rs!=null) rs.close();
if (con != null) con.close();
}

}

public void setSessionContext(SessionContext sc) {
this.sc = sc;
}

```

```

public void ejbRemove() {}
public void ejbPassivate() {}
public void ejbActivate() {}
}
//Client 부분
package db2;
import java.sql.*;
import javax.sql.*;
import javax.naming.*;
import javax.ejb.*;
import javax.rmi.*;
import db2.*;

class Db2Client {
public static void main(String[] args) {
try {
Context ctx = new InitialContext();
Object obj = ctx.lookup("db2");//Bean class와의 look up 부분
System.out.println("[look up sucess!!]");
Db2Home home =(Db2Home)PortableRemoteObject.narrow(obj,
Db2Home.class);
System.out.println("[HomeRemote interface sucess!!]");
Db2Remote db2 = home.create();
System.out.println("[Business method Call sucess!!]");
db2.query();
}catch (Exception e)
{
e.printStackTrace();
}
}
}

//JeusMain.xml
<Database>
<Vendor>db2</Vendor>
    <ExportName>db2local</ExportName>
<DatabaseName>SAMPLE</DatabaseName>

<DataSourceClassName>COM.ibm.db2.jdbc.DB2ConnectionPoolDataSource</DataSou
rceClassName>
<DataSourceName/>
<ServiceName/>
<Description/>
<NetworkProtocol/>
<Password>jeus2013</Password>
<PortNumber>0</PortNumber>
<ServerName/>
<User>db2inst1</User>
<DataSourceType>LocalXADatasource</DataSourceType>
<ConnectionPool>
<MinPoolSize>2</MinPoolSize>
<MaxPoolSize>4</MaxPoolSize>

```

```

<IncreasingStep>1</IncreasingStep>
<ResizingPeriod>600000</ResizingPeriod>
<OperationTimeout>30000</OperationTimeout>
<WaitForFreeConnection>
<WaitingTime>10000</WaitingTime>
</WaitForFreeConnection>
</ConnectionPool>
</Database>

```

다음은 Db2Client.java를 실행 하면 서버에 출력되는 화면이다.

```

[2001.09.26 01:17:35] [JNDI.CONTEXT] REBIND <db2>
[2001.09.26 01:17:35] [JMServer] REBIND <db2> <SUCCESS> Sent.
[2001.09.26 01:17:35] [Container] db2 is now bound in Naming Server
[2001.09.26 01:17:35] [JMSLocal_] <recv> CACHE DROP <db2>
[2001.09.26 01:17:35] [JMSLocal_] <recv> CACHE DROP <db2>
[2001.09.26 01:17:35] [JMSLocal_] <recv> CACHE DROP <db2>
[2001.09.26 01:17:36] [JMSLocal_] <recv> CACHE DROP <db2>
[2001.09.26 01:17:36] [EJBServer] Module Deployment Process Successful for db2local
[2001.09.26 01:17:36] -----
[2001.09.26 01:18:41] [JMServer] <ACCEPT> Local server : localhost<127.0.0.1:3106>--<9738>
[2001.09.26 01:18:41] [JmsSecurityManager] Run As guest
[InitialContext...]
[2001.09.26 01:18:42] [JNDI.CONTEXT] LOOKUP <UserTransaction>
[db2 Data Source connect...]
[2001.09.26 01:18:42] [JNDI.CONTEXT] LOOKUP <db2local>
[db2 Data Source success !!]
[2001.09.26 01:18:44] [ITMServer] a coordinator created <61.77.153.61:0:704022202:2>
[2001.09.26 01:18:46] [Coordinator] <61.77.153.61:0:704022202:2> : a local resource registered jeus.jdbc.connectionpool.LocalXAResourceWrapper@65ba24
[2001.09.26 01:18:46] [TransactionImpl] a XAResource with Local TX optimization is registered
[2001.09.26 01:18:46] [ConnectionPool] db2local : enlisted to <61.77.153.61:0:704022202:2>
*****Test1
*****Test1
[2001.09.26 01:18:46] [Terminator] <61.77.153.61:0:704022202:2> : commit starts
[2001.09.26 01:18:46] [Coordinator] <61.77.153.61:0:704022202:2> : beforeCompletion
[2001.09.26 01:18:46] [Coordinator] <61.77.153.61:0:704022202:2> : afterCompletion
[2001.09.26 01:18:46] [Coordinator] <61.77.153.61:0:704022202:2> : onePhaseCommit<3>
[2001.09.26 01:18:46] [Coordinator] <61.77.153.61:0:704022202:2> : destroyed
[2001.09.26 01:18:46] [JMServer] Connection Closed : localhost<127.0.0.1:3106>--<9738>

```

그림4-2 LocalXADataSource 실행 출력 화면

3. XADataSource에 의한 Stateful session Bean

3.1 Sybase&Oracle

```
test1.sql 생성//oracle
create table test1
(Id varchar2(20),
Passwd varchar(20));
```

```
Test2.sql 생성//sybase
Create table test2
(Id varchar2(20),
Passwd varchar(20));
```

//RemoteInterface

```
package XA;
import java.rmi.*;
import java.sql.*;
import javax.sql.*;
import javax.ejb.*;
```

```
public interface XARemote extends EJBObject {
    public void query()throws RemoteException, SQLException; ;//query()비즈니스
        메소드
}
```

//HomeInterface

```
package XA;
import java.rmi.*;
import javax.ejb.*;
```

```
public interface XAHome extends javax.ejb.EJBHome {
    public XARemote create() throws RemoteException, CreateException;
}
```

//Bean class

```
package XA;
import javax.sql.*;
import java.sql.*;
import javax.naming.*;
import javax.ejb.*;
import javax.transaction.*;
```

```
public class XABean implements SessionBean {
    public void XABean () {}
    private Connection con = null;
    private ResultSet rs=null;
    private String test1="";
    private String test2="";
    private PreparedStatement pstmt=null;
    private Statement stmt=null;
```

```

private SessionContext sc = null;
private Context ctx = null;
private DataSource xads = null;
private DataSource ds = null;
private UserTransaction ut = null;

public String ejbCreate() throws CreateException
{
try {
ctx = new InitialContext();
System.out.println("[InitialContext() success!!]");
ut = (UserTransaction)
    ctx.lookup("java:comp/UserTransaction");//Bean-Managed Transaction
    관련 JTS참고

System.out.println("[Transaction success !!]");
xads = (DataSource) ctx.lookup("XAOracle");
//JeusMain.xml에서 ExportName이oracle를look up 하는 부분

ds = (DataSource) ctx.lookup("XAsybase");
//JeusMain.xml에서 ExportName이sybase를look up 하는 부분

}
catch(Exception ex)
{
ex.printStackTrace();
}
return "success!!";
}

public void query() throws SQLException {
try {
ut.begin();//Transaction 시작
con = ds.getConnection();
stmt = con.createStatement();
rs = stmt.executeQuery("select * from test1");
while (rs.next())
{
test1 = rs.getString("Id");
test2 = rs.getString("Passwd");
System.out.println("*****"+test1);
System.out.println("*****"+test2);
}

con = xads.getConnection();
PreparedStatement pstmt = con.prepareStatement("insert into test2 values
( ?, ?)");
pstmt.setString(1, test1);
pstmt.setString(2, test2);
pstmt.executeUpdate();

```

```

        ut.commit();//Transaction 성공
    } catch (Exception e)
    {
        e.printStackTrace();
        try {
            ut.rollback();//Transaction 실패
        } catch (Exception ee)
        {
            ee.printStackTrace();
        }
    } finally
    {
        if (pstmt != null) pstmt.close();
        if( rs!=null) rs.close();
        if (con != null) con.close();
    }
}

public void setSessionContext(SessionContext sc) {
    this.sc = sc;
}

public void ejbRemove() {}
public void ejbPassivate() {}
public void ejbActivate() {}

}

//Client class
package XA;
import javax.sql.*;
import java.sql.*;
import javax.naming.*;
import javax.ejb.*;
import javax.rmi.*;
import XA.*;

class XAClient {
    public static void main(String[] args) {
        try {
            Context ctx = new InitialContext();
            Object obj = ctx.lookup("xaSource");//Bean class와의 look up 부분
            System.out.println("[look up sucess!!]");
            XAHome home
                =(XAHome)PortableRemoteObject.narrow(obj, XAHome.class);

            XARemote xa = home.create();
            System.out.println("[HomeRemote interface sucess!!]");
            xa.query();
            System.out.println("[Business method Call sucess!!]");
        }
    }
}

```

```

        catch (Exception e)
        {
            e.printStackTrace();
        }
    }
}

```

```

//JeusMain.xml
//Oracle XA configure
<Database>
<Vendor>oracle</Vendor>
<ExportName>XAOracle</ExportName>
<DatabaseName>ora8i</DatabaseName>
<DataSourceClassName>oracle.jdbc.xa.client.OracleXADataSource</DataSourceC
lassName>
<DataSourceName>oracle.jdbc.xa.client.OracleXADataSource</DataSourceName>
<ServiceName/>
<Description/>
<NetworkProtocol/>
<Password>tiger</Password>
<PortNumber>1521</PortNumber>
<ServerName>jeus</ServerName>
<User>scott</User>
<DriverType>thin</DriverType>
<DataSourceType>XADataSource</DataSourceType>
<ConnectionPool>
<MinPoolSize>1</MinPoolSize>
<MaxPoolSize>2</MaxPoolSize>
<IncreasingStep>1</IncreasingStep>
<ResizingPeriod>1</ResizingPeriod>
<OperationTimeout>30000</OperationTimeout>
<WaitForFreeConnection>
<WaitingTime>10000</WaitingTime>
</WaitForFreeConnection>
</ConnectionPool>
</Database>
//Sybase XA configure

<Database>
<Vendor>sybase</Vendor>
<ExportName>XAsybase</ExportName>
<DatabaseName>testdb</DatabaseName>
<DataSourceClassName>com.sybase.jdbc2.jdbc.SybXADataSource</DataSourceClas
sName>
<DataSourceName>com.sybase.jdbc2.jdbc.SybXADataSource</DataSourceName>
<ServiceName/>
<Description>sybase</Description>
<NetworkProtocol>Tds</NetworkProtocol>
<Password>sybase</Password>
<PortNumber>4100</PortNumber>
<ServerName>jeus30</ServerName>
<User>sa</User>
<DataSourceType>XADataSource</DataSourceType>

```

```

<ConnectionPool>
<MinPoolSize>10</MinPoolSize>
<MaxPoolSize>50</MaxPoolSize>
<IncreasingStep>5</IncreasingStep>
<ResizingPeriod>10</ResizingPeriod>
<OperationTimeout>30000</OperationTimeout>
<WaitForFreeConnection>
<WaitingTime>10000</WaitingTime>
</WaitForFreeConnection>
</ConnectionPool>
</Database>

```

다음은 Db2Client.java를 실행 하면 서버에 출력되는 화면이다.

```

[2001.09.26 02:22:20] [JMSServer] REBIND <xaSource> <SUCCESS> Sent.
[2001.09.26 02:22:20] [JMSLocal_] <recv> CACHE DROP <xaSource>
[2001.09.26 02:22:20] [JMSLocal_] <recv> CACHE DROP <xaSource>
[2001.09.26 02:22:20] [JMSLocal_] <recv> CACHE DROP <xaSource>
[2001.09.26 02:22:20] [JMSLocal_] <recv> CACHE DROP <xaSource>
[2001.09.26 02:22:20] [Container] xaSource is now bound in Naming Server
[2001.09.26 02:22:21] [EJBServer] Module Deployment Process Successful for ayhas
aXA
[2001.09.26 02:22:21] -----
[2001.09.26 02:22:35] [JMSServer] <ACCEPT> Local server : localhost(127.0.0.1:37
95)---<9738>
[2001.09.26 02:22:35] [JmsSecurityManager] Run As guest
[InitialContext<> success!!]
[2001.09.26 02:22:36] [JNDI.CONTEXT] LOOKUP <UserTransaction>
[Transaction success !!]
[2001.09.26 02:22:36] [JNDI.CONTEXT] LOOKUP <XAOracle>
[2001.09.26 02:22:36] [JNDI.CONTEXT] LOOKUP <XAayhas>
[2001.09.26 02:22:36] [TMServer] a coordinator created <61.77.153.61:0:71071836
1:1>
[2001.09.26 02:22:37] [Coordinator] <61.77.153.61:0:710718361:1> : a local reso
uce registered jms.jdbc.connectionpool.XAResourceWrapper@7e5619
[2001.09.26 02:22:37] [ConnectionPool] XAOracle : enlisted to <61.77.153.61:0:7
10718361:1>
*****get1
*****get2
[2001.09.26 02:22:37] [Coordinator] <61.77.153.61:0:710718361:1> : a local reso
uce registered jms.jdbc.connectionpool.XAResourceWrapper@5c2043
[2001.09.26 02:22:37] [ConnectionPool] XAayhas : enlisted to <61.77.153.61:0:7
10718361:1>
[2001.09.26 02:22:37] [Terminator] <61.77.153.61:0:710718361:1> : commit starts
[2001.09.26 02:22:37] [Coordinator] <61.77.153.61:0:710718361:1> : beforeComple
ion
[2001.09.26 02:22:37] [Coordinator] <61.77.153.61:0:710718361:1> : prepare resu
<0>
[2001.09.26 02:22:37] [Terminator] global decision of <61.77.153.61:0:710718361
1> is 0
[2001.09.26 02:22:37] [Coordinator] <61.77.153.61:0:710718361:1> : afterComple
on
[2001.09.26 02:22:37] [Coordinator] <61.77.153.61:0:710718361:1> : commit<3>
[2001.09.26 02:22:37] [Coordinator] <61.77.153.61:0:710718361:1> : destroyed
[2001.09.26 02:22:37] [JMSServer] Connection Closed : localhost(127.0.0.1:3795)
---<9738>

```

그림4-3 XADatasource 실행 출력 화면

위의 과정에서 EJB deploy 하는 과정은 *deployment Tool*를 참고